

Ch. 2

Représentation de nombres naturels et entiers

B. Quoitin
(bruno.quoitin@umons.ac.be)

Table des Matières

Notation des Nombres

Représentation décimale

- Notation Positionnelle Généralisée
- Changement de Base

Nombres dans un Ordinateur

- Représentation des naturels
- Représentation des entiers

Représentation des entiers

Objectifs

- Dans un monde digital, tout doit être encodé. Ce chapitre s'intéresse à l'encodage des nombres entiers.
 - Formaliser la manière dont nous *représentons* les nombres en utilisant la **notation positionnelle décimale** (une représentation apprise à l'école primaire).
 - Découvrir des représentations non-décimales de nombres reposant sur la **notation positionnelle généralisée**.
 - Maîtriser les représentations non-décimales utiles en informatique sont le **binaire**, l'**octal** et l'**hexadécimal**.
 - Apprendre à **convertir** entre les différentes représentations.
- Des chapitres ultérieurs concernent l'encodage d'autres informations telles que des caractères, des nombres réels et des instructions processeur.

Représentation décimale

Chiffres/symboles

- La représentation décimale se base sur l'utilisation de 10 symboles, les chiffres décimaux : “0”, “1”, “2”, “3”, “4”, “5”, “6”, “7”, “8” et “9”. A chaque chiffre correspond un nombre entre 0 et 9. Ainsi,
 - le chiffre « 0 » représente le nombre 0,
 - le chiffre « 1 » représente le nombre 1,
 - et ainsi de suite.
- Attention, un chiffre est un *symbole* permettant de représenter un nombre. Il est important de réaliser que **chiffre** (*représentation*) et **nombre** (*représenté*) sont des concepts différents.

Séquence et position

- Pour représenter des nombres plus grands que 9, une séquence de chiffres est utilisée. La position d'un chiffre dans cette séquence est importante car elle donne un poids à ce chiffre. Il s'agit d'une représentation dit « **positionnelle** ».

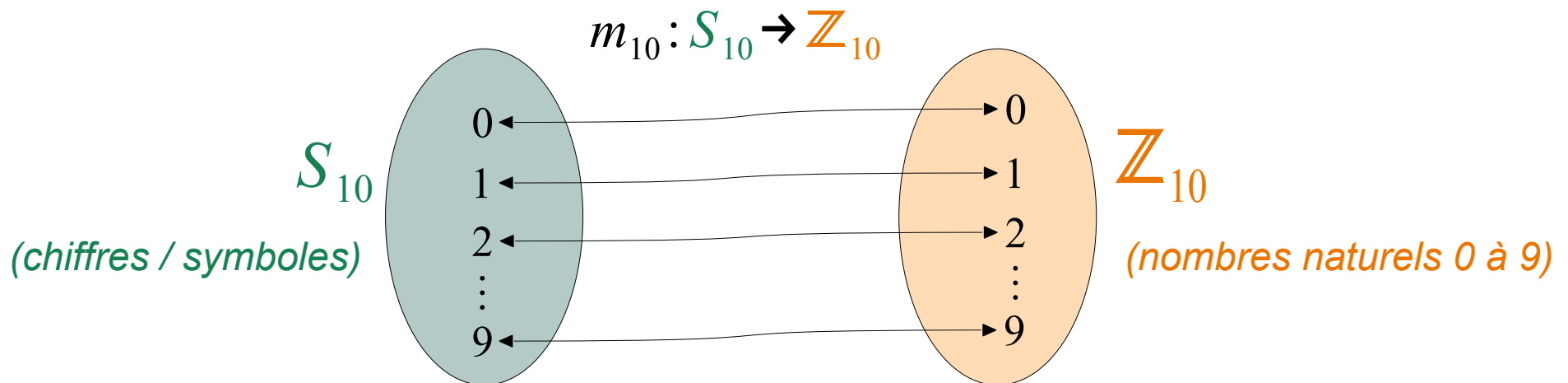
Représentation décimale

Tentative de formalisation

- Plus formellement, la représentation décimale utilise un ensemble de 10 chiffres

$$S_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

- Une bijection m_{10} fait correspondre chaque chiffre de S_{10} à un unique nombre naturel compris entre 0 et 9 inclus.



Note : en représentation décimale, nous confondrons souvent les chiffres décimaux avec les nombres qu'ils représentent lorsqu'il n'y a pas d'ambiguïté.

Représentation décimale

Tentative de formalisation

- Un nombre naturel x est représenté par un **mot** w composé de N chiffres w_i pris dans S_{10}

$$w = w_{N-1} w_{N-2} \dots w_1 w_0$$

- Interpréter une représentation w consiste à déterminer le nombre x représenté. On utilise à cet effet la formule suivante. Le chiffre w_i de position i est associé au **poids** 10^i où 10 est la **base** du système.

$$x = \sum_{i=0}^{N-1} m_{10}(w_i) \cdot 10^i$$

- Exemple : que représente $w = \text{« 32768 »}$ ($w_0 = 8, w_1 = 6, \dots, w_4 = 3$) ?

$$\begin{aligned} x &= \sum_{i=0}^{N-1} w_i \cdot 10^i \\ &= m_{10}(3) \cdot 10^4 + m_{10}(2) \cdot 10^3 + m_{10}(7) \cdot 10^2 + m_{10}(6) \cdot 10^1 + m_{10}(8) \cdot 10^0 \\ &= 3 \cdot 10^4 + 2 \cdot 10^3 + 7 \cdot 10^2 + 6 \cdot 10 + 8 = 32768 \end{aligned}$$

Représentation décimale

Poids des chiffres

- Chaque chiffre d'un mot w est associé à un poids en fonction de sa position.
 - le chiffre le plus à gauche est appelé **chiffre de poids (le plus) fort**.
 - le chiffre le plus à droite est appelé **chiffre de poids (le plus) faible**.

$$w = w_{N-1} w_{N-2} \dots w_1 w_0$$


Diagram illustrating the weight of digits in a decimal number. The number is represented as $w = w_{N-1} w_{N-2} \dots w_1 w_0$. A red arrow points left from the rightmost digit, labeled "poids fort", and a blue arrow points right from the leftmost digit, labeled "poids faible".

Chiffres significatifs

- Les chiffres significatifs d'une représentation sont les chiffres qui suivent le chiffre non nul de poids le plus fort, y compris ce dernier. Le nombre représenté ne change pas si on supprime les chiffres non significatifs.
- Exemple : dans 004302, il y a 4 chiffres significatifs, 4302.

Représentation décimale

Nombre de mots différents

- Il existe 10^N mots de N chiffres. Il est donc possible avec N chiffres de représenter 10^N nombres différents.

$$w = w_{N-1} w_{N-2} \dots w_1 w_0$$

(10 possibilités pour w_{N-1}) * ... * (10 possibilités pour w_1) * (10 possibilités pour w_0)

Intervalle représentable

- L'intervalle de nombres représentables avec N chiffres est égal à

$$[0, 10^N - 1]$$

Représentation décimale

Taille de mot

- Etant donné un nombre naturel x , quelle doit être la valeur minimale de $N \in \mathbb{N}$ pour pouvoir représenter x ?
- Pour que x soit représentable sur N bits, il faut qu'il soit inférieur ou égal au plus grand nombre représentable

$$x \leq 10^N - 1$$

$$x + 1 \leq 10^N$$

$$\log_{10}(x + 1) \leq \log_{10}(10^N) = N$$

- Le plus petit nombre de chiffres nécessaires pour représenter x est donc obtenu comme

$$N \geq \lceil \log_{10}(x + 1) \rceil$$

- Exemple : combien de chiffres significatifs sont nécessaires pour représenter le nombre $x = 1234$?

Table des Matières

Notation des Nombres

- Représentation décimale

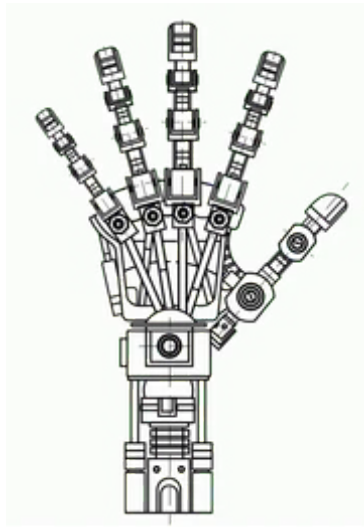
Notation Positionnelle Généralisée

- Changement de Base

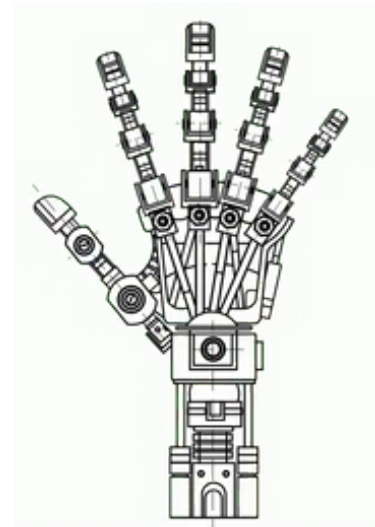
Nombres dans un Ordinateur

- Représentation des naturels
- Représentation des entiers

Notation Positionnelle Généralisée



**Les ordinateurs
n'ont pas dix doigts !**



Objectif

- Nous sommes familiarisés avec la notation décimale (base 10). Les ordinateurs utilisent la notation binaire (base 2).
- Deux symboles/chiffres, notés 0 et 1, qui peuvent être implémentés par 2 états d'un système électronique.
- La notation positionnelle généralisée permet de représenter des nombres en utilisant d'autres bases.

Notation Positionnelle Généralisée

Généralisation

- Soit $k \geq 2$ la base utilisée.
- Soit S_k un ensemble de k symboles différents. $S_k = \{s_0, \dots, s_{k-1}\}$
- Soit m_k une bijection qui fait correspondre chaque symbole de S_k à un nombre compris entre 0 et $k-1$.

$$m_k : S_k \rightarrow \mathbb{Z}_k : s_k \rightarrow k$$

- Un nombre naturel x est représenté par un mot w de N symboles pris dans S_k .

$$w = w_{N-1} w_{N-2} \dots w_1 w_0 \quad (\forall i, w_i \in S_k)$$

- Le nombre x représenté par la séquence w est obtenu par la formule généralisée

$$x = \sum_{i=0}^{N-1} m_k(w_i) \cdot k^i$$

Notation Positionnelle Généralisée

Exemples

- $k = 10$, chiffres décimaux

- Identique à la notation positionnelle décimale.
- Ensemble des symboles : $S_{10} = \{0, \dots, 9\}$
- Bijection : $m_{10} : S_{10} \rightarrow \mathbb{Z}_{10} : \{(\overset{\text{symbole}}{0}, \overset{\text{nombre}}{0}), (\overset{\text{symbole}}{1}, \overset{\text{nombre}}{1}), (\overset{\text{symbole}}{2}, \overset{\text{nombre}}{2}), \dots, (\overset{\text{symbole}}{9}, \overset{\text{nombre}}{9})\}$

- $k = 10$, lettres

- Nous pourrions utiliser des symboles différents des chiffres décimaux pour travailler en base 10. Par exemple, les lettres A à J
- Ensemble des symboles : $S_{10} = \{A, B, C, D, E, \dots, J\}$
- Bijection : $m_{10} : S_{10} \rightarrow \mathbb{Z}_{10} : \{(A, 0), (B, 1), (C, 2), \dots, (I, 8), (J, 9)\}$
- Quel nombre représente le mot « CHD » ?

$$\begin{aligned} x &= \sum_{i=0}^2 m_{10}(w_i) \cdot 10^i \\ &= m_{10}(C) \cdot 10^2 + m_{10}(H) \cdot 10^1 + m_{10}(D) \\ &= 200 + 70 + 3 = 273 \end{aligned}$$

Notation Positionnelle Généralisée

Représentation binaire

- Base $k=2$
- En binaire, les symboles sont appelés **bits** (contraction de « *binary digits* »). Un groupe de 8 bits est appelé **octet** (« *byte* » en anglais).
- Ensemble des symboles : $S_2 = \{0, 1\}$
- Bijection : $m_2 : S_2 \rightarrow \mathbb{Z}_2 : \{(0, 0), (1, 1)\}$
- Quel nombre représente la séquence « 110101 » ?

$$\begin{aligned}x &= \sum_{i=0}^5 m_2(w_i) \cdot 2^i \\&= 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\&= 1 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 53\end{aligned}$$

Notation Positionnelle Généralisée

Représentation hexadécimale

- Base $k=16$
- La représentation hexadécimale est fort utilisée en informatique car la conversion du / vers le binaire est aisée. La représentation hexadécimale est également plus compacte qu'en binaire.
- Ensemble des symboles : $S_{16} = \{0, 1, \dots, 8, 9, A, B, \dots, F\}$
- Bijection : $m_{16} : S_{16} \rightarrow \mathbb{Z}_{16} : \{(0, 0), \dots, (9, 9), (A, 10), \dots, (F, 15)\}$
- Note : pour indiquer que l'on travaille en base 16, les nombres hexadécimaux sont parfois notés avec le préfixe "0x". Par exemple, « CAFE » sera souvent noté 0xCAFE.
- Quel nombre représente la séquence « F3AD » ?

$$\begin{aligned} x &= \sum_{i=0}^3 m_{16}(w_i) \cdot 16^i \\ &= m_{16}(F) \cdot 16^3 + m_{16}(3) \cdot 16^2 + m_{16}(A) \cdot 16 + m_{16}(D) \\ &= 15 \cdot 4096 + 3 \cdot 256 + 10 \cdot 16 + 13 = 62381 \end{aligned}$$

Notation Positionnelle Généralisée

Exemples de nombres représentés avec différentes bases

Décimal	Binaire	Octal	Hexadécimal
0	0	0	0
2	10	2	2
8	1000	10	8
11	1011	13	B
20	10100	24	14
42	101010	52	2A
117	1110101	165	75
228	11100100	264	E4

Notation Positionnelle Généralisée

Intervalle de nombres représentables

- Un mot w de longueur N peut représenter les nombres naturels dans l'intervalle

$$[0, k^N - 1]$$

- Exemple : en binaire, avec $N=4$, cet intervalle est $[0, 15]$.

Longueur minimale de mot

- Par un raisonnement similaire à celui utilisé en décimal, on peut montrer que la longueur N du plus petit mot permettant de représenter un naturel x est donnée par

$$N \geq \lceil \log_k (x+1) \rceil$$

- Exemple : combien de symboles significatifs sont nécessaires pour représenter le nombre 1234 en hexadécimal ?

$$N \geq \lceil \log_{16} (1234+1) \rceil = \lceil \log_{16} (1235) \rceil = \lceil 2,566... \rceil = 3$$

Table des Matières

Notation des Nombres

- Représentation décimale
- Notation Positionnelle Généralisée

 Changement de Base

Nombres dans un Ordinateur

- Représentation des naturels
- Représentation des entiers

Changement de base

Objectifs

- **Changement de base**
 - Comment convertir la représentation d'un nombre d'une base dans une autre ?
 - Par exemple, comment passer de la base 10 à la base 2 et vice-versa ? comment convertir entre les représentations binaire, hexadécimale et octale ?
- **Trucs / astuces / algorithmes de conversion**
 - binaire/hexadécimal $\rightarrow$ décimal
 - binaire $\leftrightarrow$ hexadécimal $\leftrightarrow$ octal
 - décimal $\rightarrow$ binaire/hexadécimal

Changement de base

Conversion base $k \rightarrow$ décimal

- La conversion est « immédiate ». Il suffit d'appliquer la formule donnant le nombre représenté par une séquence w en base k pour obtenir sa représentation en décimal (pourquoi?).

$$x = \sum_{i=0}^{N-1} m_k(w_i) \cdot k^i$$

- Exemples

- « 1011010 » en binaire $\rightarrow 2^6 + 2^4 + 2^3 + 2^1 = 90$
- « 50 » en hexadécimal $\rightarrow 5 \times 16 = 80$
- « 13F » en hexadécimal $\rightarrow 1 \times 256 + 3 \times 16 + 15 = 319$

Changement de base

Conversion décimal $\rightarrow$ base k

- Il est possible d'utiliser la division euclidienne de façon répétée.
- La séquence w qui représente le nombre x en base k vérifie l'équation

$$\begin{aligned}x &= \sum_{i=0}^{N-1} m_k(w_i) \cdot k^i \\&= m_k(w_0) \cdot k^0 + m_k(w_1) \cdot k^1 + \dots + m_k(w_{N-1}) \cdot k^{N-1}\end{aligned}$$

- En factorisant k chaque fois que cela est possible, cette équation peut être ré-écrite comme suit

$$x = m_k(w_0) + k \cdot \left(m_k(w_1) + k \cdot \left(\dots + k \cdot m_k(w_{N-1}) \right) \right)$$

- En effectuant la division euclidienne de x par la base k , on obtient
 - **quotient:** $q = x/k = m_k(w_1) + k \cdot \left(\dots + k \cdot m_k(w_{N-1}) \right)$
 - **reste:** $r = x \bmod k = m_k(w_0) \xrightarrow{m_k^{-1}} w_0$

Changement de base

Conversion décimal $\rightarrow$ non-décimal (suite)

- L'application répétée de la division sur le quotient obtenu permet d'obtenir successivement $w_0, \dots, w_{N-1}$

$$x = m_k(w_0) + k \cdot (m_k(w_1) + k \cdot (\dots + k \cdot m_k(w_{N-1})))$$

$$q = m_k(w_1) + k \cdot (\dots + k \cdot m_k(w_{N-1}))$$

$$r = m_k(w_0)$$

La même opération
est répétée en partant
de $x=q$

$$x = m_k(w_1) + k \cdot (\dots + k \cdot m_k(w_{N-1}))$$

$$q = m_k(w_2) + k \cdot (\dots + k \cdot m_k(w_{N-1}))$$

$$r = m_k(w_1)$$

$\vdots$

$$x = m_k(w_{N-1})$$

$$q = 0$$

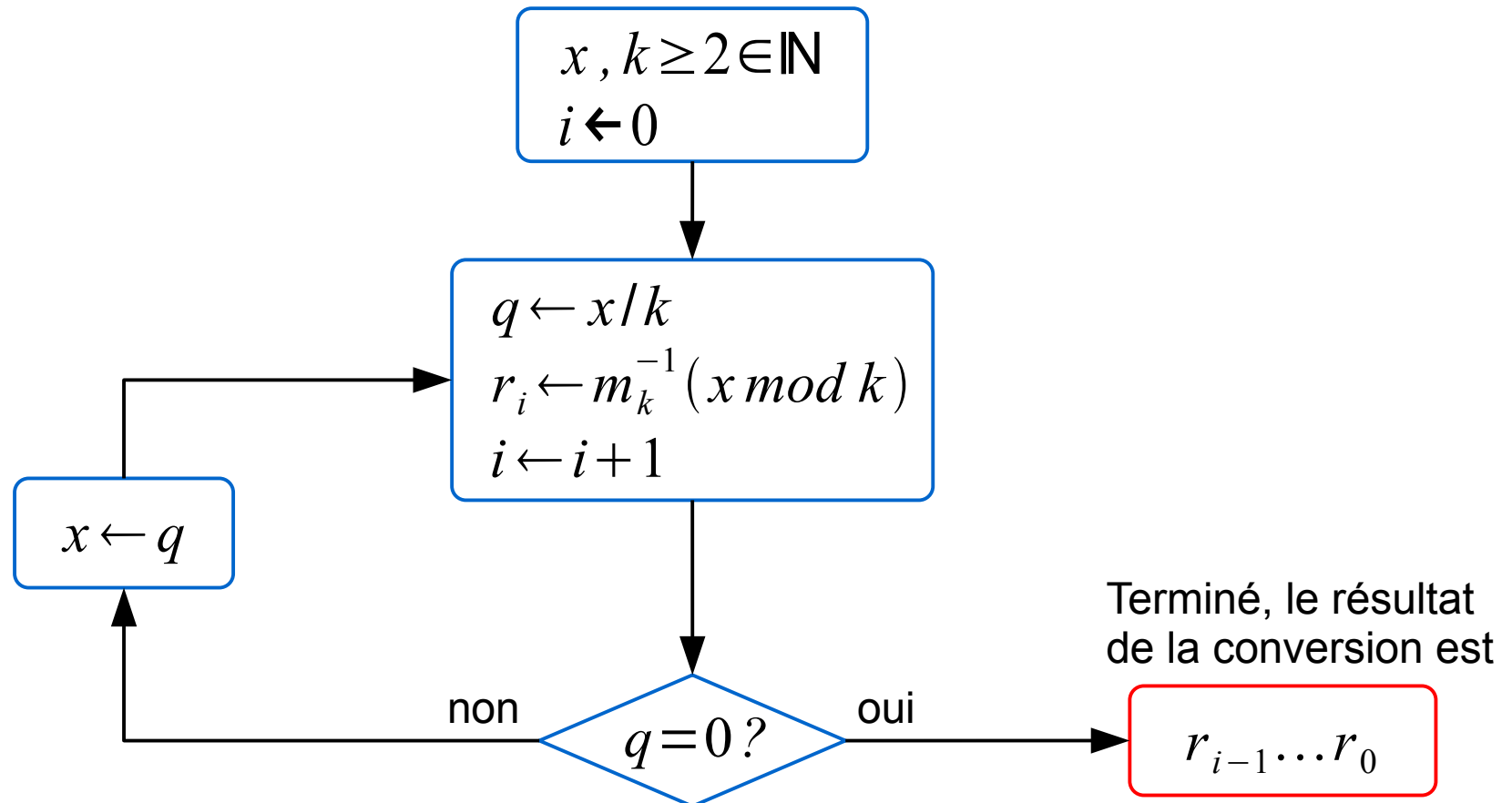
$$r = m_k(w_{N-1})$$

La conversion est terminée
lorsque le quotient vaut 0

Changement de base

Algorithme

- La conversion de décimal en non-décimal peut être réalisée à l'aide de l'algorithme suivant :



Changement de base

Implémentation en Python3

```
import sys

SYMBOLS = "0123456789ABCDEF"

base_s = input("Entrez la base: ")
base = int(base_s, 10)
if (base < 2) or (base > 16):
    sys.exit("Erreur: la base doit etre " \
            "comprise entre 2 et 16")

x_s = input("Entrez le nombre: ")
x = int(x_s, 10)
if (x < 0):
    sys.exit("Erreur: le nombre doit etre positif")

word = "0" if (x == 0) else ""
while (x != 0):
    q = x // base
    r = x % base
    word = "%c%s" % (SYMBOLS[r], word)
    x = q

print("en base %d : %s" % (base, word))
```


Changement de base

Exemple

- Quelle est la représentation binaire du nombre 1984 ?

x	quotient	reste
1984 / 2	992	0
992 / 2	496	0
496 / 2	248	0
248 / 2	124	0
124 / 2	62	0
62 / 2	31	0
31 / 2	15	1
15 / 2	7	1
7 / 2	3	1
3 / 2	1	1
1 / 2	0	1

bit de poids faible

11111000000₂

bit de poids fort

Changement de base

Exemple

- Quelle est la représentation hexadécimale du nombre 1984 ?

x	quotient	reste
1984 / 16	124	0
124 / 16	7	12
7 / 16	0	7

Diagram illustrating the conversion of 1984 to hexadecimal (7C0₁₆):

- The remainder 0 from the first division is labeled "symbole de poids faible" (least significant digit) and is associated with the weight m_{16}^{-1} .
- The remainder 12 from the second division is labeled "symbole de poids fort" (most significant digit) and is associated with the weight m_{16}^{-1} .

The final hexadecimal representation is 7C0₁₆.

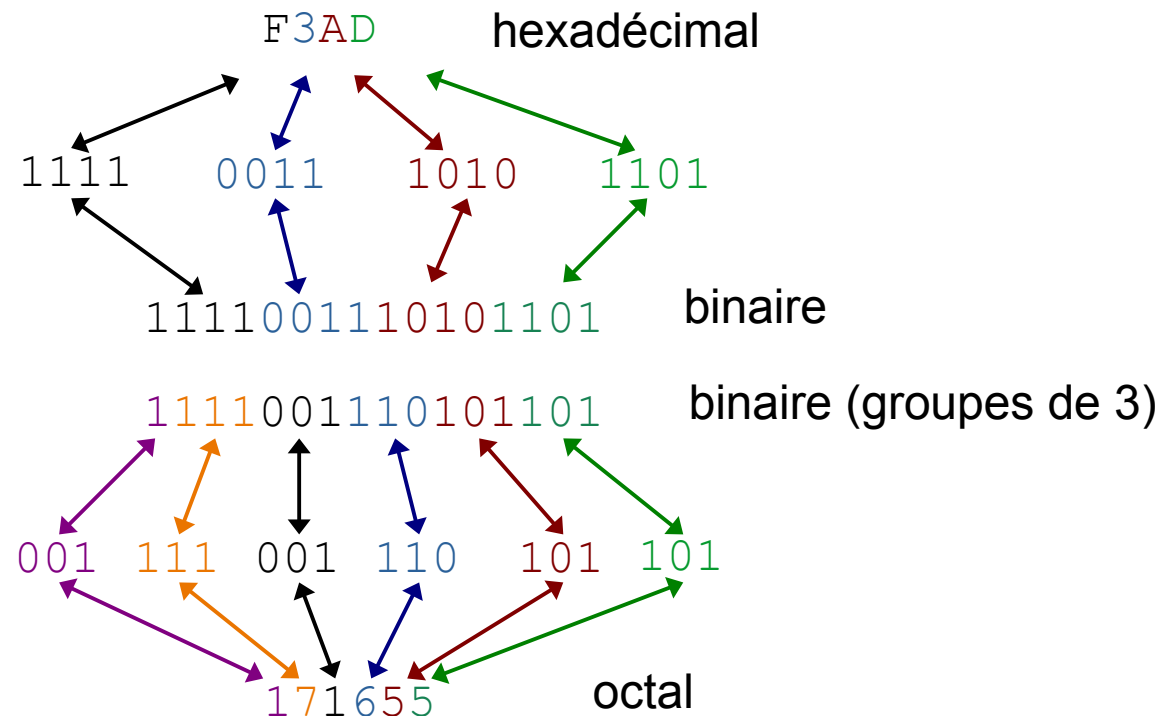
$$m_{16}^{-1} : \mathbb{Z}_{16} \rightarrow S_{16} : \{(0, 0), \dots, (9, 9), (10, A), \dots, (15, F)\}$$

Changement de base

Conversion binaire ↔ hexadécimal/octal

- Les représentations octales et hexadécimales utilisent des bases qui sont des exposants de 2. Cela signifie qu'un symbole octal ou hexadécimal correspond directement à plusieurs bits.
 - Octal, $k = 8 = 2^3 \rightarrow 1$ symbole octal = 3 bits
 - Hexadécimal, $k = 16 = 2^4 \rightarrow 1$ symbole hexadécimal = 4 bits

- Exemple



Hex	Dec	Bin
D	13	1101
A	10	1010
3	3	0011
F	15	1111

Table des Matières

Notation des Nombres

- Représentation décimale
- Notation Positionnelle Généralisée
- Changement de Base

Nombres dans un Ordinateur

Représentation des naturels

- Représentation des entiers

Nombres dans un Ordinateur

Représentation binaire

- La majorité des ordinateurs d'aujourd'hui représentent des nombres **sous forme binaire**. Les bits 0 et 1 sont manipulés à l'intérieur d'un processeur sous forme de deux tensions électriques différentes.
 - par exemple : $0 \leftrightarrow 0\text{ V}$ et $1 \leftrightarrow 5\text{ V}$
- Un processeur utilise des **mots binaires**, le plus souvent **de taille fixe**, pour représenter des nombres. Les tailles usuelles de ces mots sont des exposants de 2, plus spécifiquement 8, 16, 32 et 64 bits.



- Les opérations arithmétiques sont effectuées sur la représentation des nombres sous forme de mots binaires. Dans un processeur, ces opérations sont réalisées par l'**Unité Arithmétique et Logique** (ALU).
- Les processeurs utilisent des conventions différentes pour représenter les **naturels** (dits « *non-signés* ») et les **entiers** (dits « *signés* »). Nous les discutons séparément dans ce chapitre.

Représentation des Naturels

Principe

- Les *nombre naturels* (*entiers non signés*) sont représentés en utilisant la **notation positionnelle**, dans des **mots binaires de taille fixe**. Un mot de N bits permet de représenter des valeurs dans l'intervalle

$$[0, 2^N - 1]$$

- Chaque processeur est capable de manipuler des mots de tailles spécifiques.
- Certains langages de programmation permettent de contrôler la taille de représentation en associant des types spécifiques aux variables.
 - Exemple, en langage C sur un processeur 32 bits:

N	Type	Intervalle de nombres
8	unsigned char	[0, 255]
16	unsigned short	[0, 65535]
32	unsigned int	[0, 4294967295]
64	unsigned long int	[0, 18446744073709551615]

Table des Matières

Notation des Nombres

- Représentation décimale
- Notation Positionnelle Généralisée
- Changement de Base

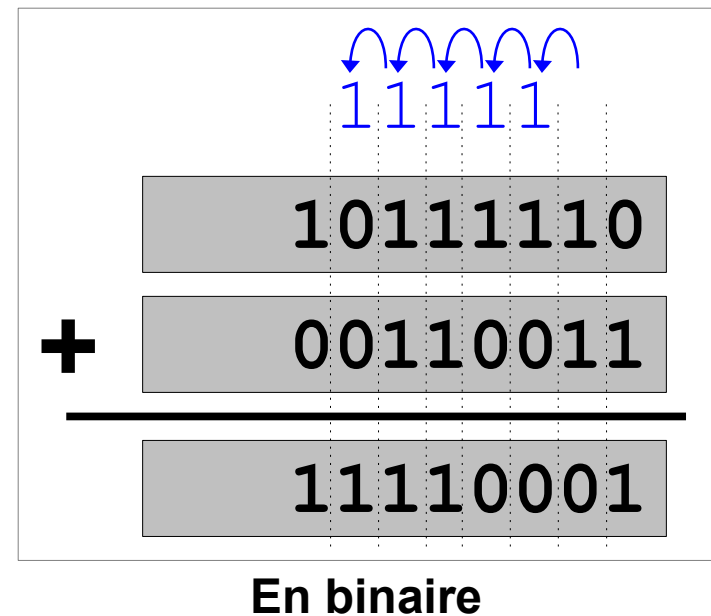
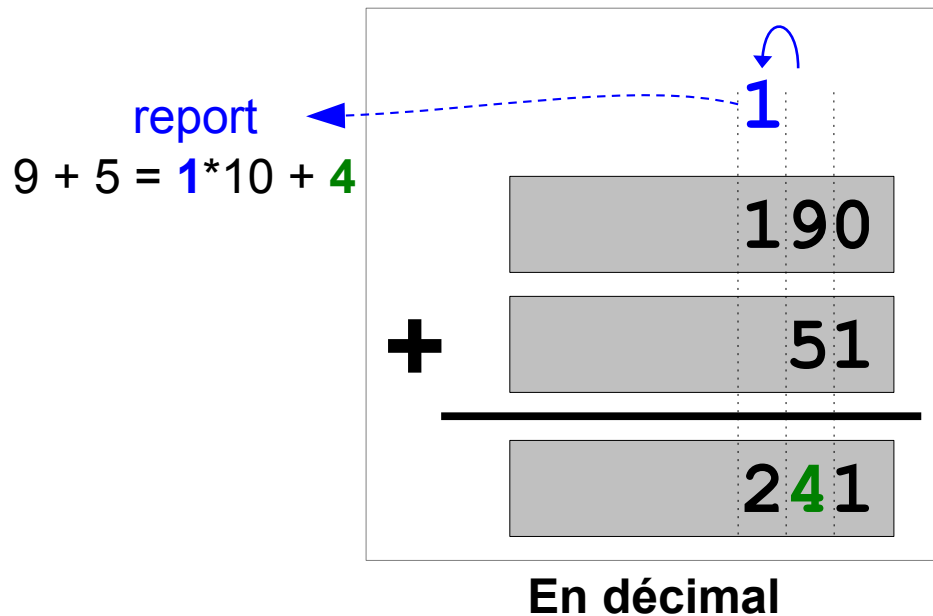
Nombres dans un Ordinateur

- Représentation des naturels
 - ➡ Addition
 - Soustraction
 - Décalages à gauche et à droite
 - Multiplication
 - Division
- Représentation des entiers

Opérations sur les Naturels

Addition en décimal vs en binaire

- Pour effectuer l'addition de deux nombres naturels x et y , nous utilisons un algorithme qui travaille sur la représentation décimale/binaire de ces nombres.
- L'algorithme procède **rang par rang**, en commençant par les chiffres/bits de poids faible. A chaque rang i ($0 \leq i < N$), un chiffre/bit s_i de la somme est produit sur base des chiffres/bits x_i et y_i . Un **report** (*carry*) peut être propagé au rang suivant.



Opérations sur les Naturels

Exemple en langage C

```
#include <stdio.h>

int main() {
    unsigned char x = 190;
    unsigned char y = 141;
    unsigned char z = x + y;
    printf("%u\n", z);
    return 0;
}
```

→ Qu'affichera le programme ?

Notes :

pour compiler l'exemple ci-dessus (main.c), utilisez la commande

```
gcc -Wall -Werror -o main main.c
```

pour l'exécuter, utilisez

```
./main
```

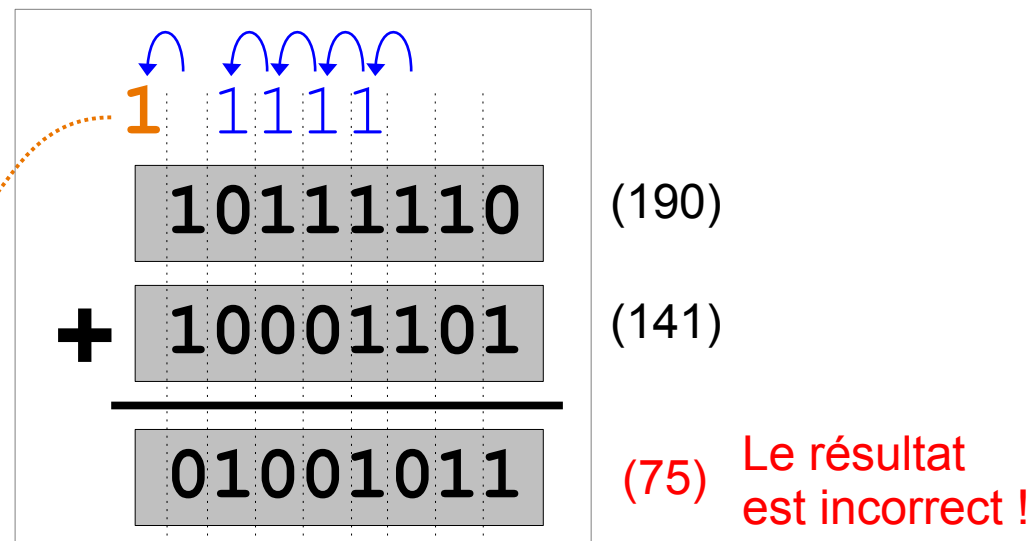
Opérations sur les Naturels

Dépassement (overflow)

- L'addition a lieu **sur N bits**.
 - Un mot peut représenter des nombres compris entre 0 et $2^N - 1$.
 - La somme de deux nombres est comprise entre 0 et $2 \cdot (2^N - 1) = 2^{N+1} - 2$
- Si le résultat n'est pas représentable sur N bits, i.e. si $x + y > 2^N - 1$, alors il y a **dépassement** (overflow). Seuls les N bits de poids faible du résultat sont conservés. Lorsqu'un dépassement a lieu, il y a un report au dernier rang et le résultat calculé est incorrect.

- Exemple ($N=8$)

Le bit de report de rang $N-1$ indique qu'il y a dépassement.



Opérations sur les Naturels

Addition modulo 2^N

- Lors d'un dépassement, le bit de report est ignoré. Par conséquent, l'addition de nombres naturels représentés avec N bits dans un ordinateur est une **addition « modulo 2^N »**.
- Si on note l'addition de naturels (addition non-signée) par l'opérateur $+_{ns}$, on a que

$$x +_{ns} y = \begin{cases} x + y & \text{si } x + y < 2^N \\ x + y - 2^N & \text{si } x + y \geq 2^N \end{cases}$$

- Exemple ($N=8$) : $190 +_{ns} 141 = 331 - 256 = 75$
- Pour les matheux : l'ensemble des nombres naturels représentables avec N bits muni de l'addition $+_{ns}$ forme un groupe commutatif.
Question subsidiaire : comment le prouver ?

Table des Matières

Notation des Nombres

- Représentation décimale
- Notation Positionnelle Généralisée
- Changement de Base

Nombres dans un Ordinateur

- Représentation des naturels
 - Addition
 - ➡ Soustraction
 - Décalages à gauche et à droite
 - Multiplication
 - Division
- Représentation des entiers

Opérations sur les Naturels

Soustraction en décimal vs en binaire

- Comme pour l'addition, l'algorithme de soustraction fonctionne rang par rang. La différence est qu'à chaque rang, une opération de soustraction est effectuée et qu'une **retenue** (*borrow*) est effectuée. Celle-ci est soustraite au rang suivant.

reports
(borrow)

$$\begin{array}{r} - \\ 210 \\ - 109 \\ \hline 101 \end{array}$$

En décimal

$$\begin{array}{r} - \\ 11010010 \\ - 01101101 \\ \hline 01100101 \end{array}$$

En binaire

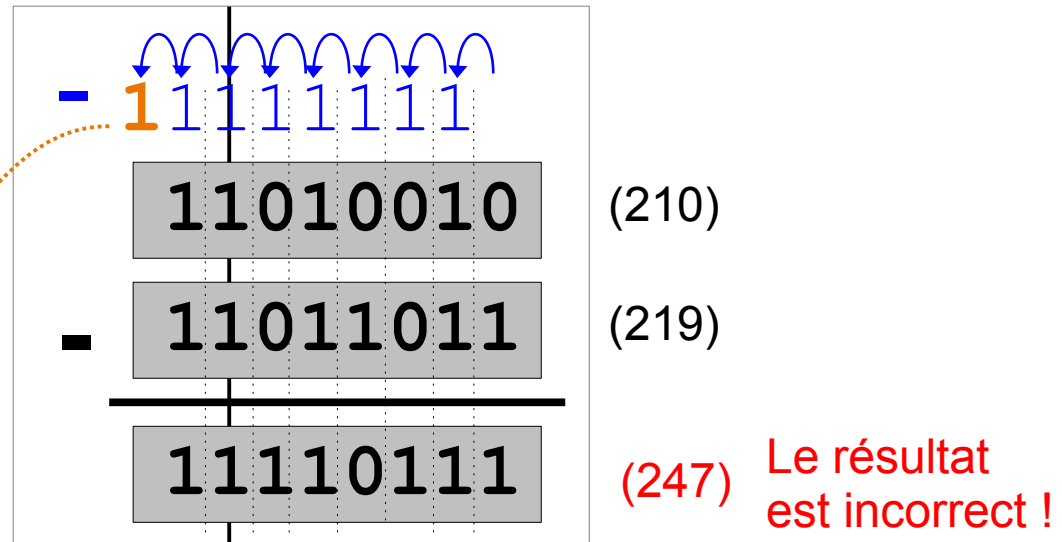
Opérations sur les Naturels

Résultat négatif (*underflow*)

- La soustraction de nombres naturels peut ne pas avoir un résultat naturel, si $x < y$, alors $x - y < 0$ n'est pas représentable.
- Dans cette situation, il existe un report au dernier rang et le résultat est incorrect.

- Exemple ($N = 8$)

Il y a un **report final non nul**.
Le résultat est négatif
(pas un naturel).



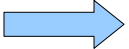
- Comme on travaille sur $N=8$ bits, le résultat vaut 247 au lieu de -9.
L'explication est similaire à celle de l'addition modulo 2^N : $(247 = 2^N - 9 = 2^8 - 9 = 256 - 9)$

Table des Matières

Notation des Nombres

- Représentation décimale
- Notation Positionnelle Généralisée
- Changement de Base

Nombres dans un Ordinateur

- Représentation des naturels
 - Addition
 - Soustraction
 -  Décalages à gauche et à droite
 - Multiplication
 - Division
- Représentation des entiers

Opérations sur les Naturels

Décalages à gauche et à droite

- Les opérations de décalage à gauche (notée « $\ll$ ») et à droite (notée « $\gg$ ») ne sont pas à proprement parler des opérations arithmétiques. Elles permettent cependant d'effectuer efficacement des multiplications et divisions entières par des exposants de 2.

Décalage à gauche

00010101

(21)

$\ll$

2

01010100

(84)

y bits de poids fort
sont perdus.

y bits de poids faible
valant 0 sont insérés.

Décalage à droite

01111011

(123)

$\gg$

3

00001111

(15)

y bits de poids fort
valant 0 sont insérés.

y bits de poids faible
sont perdus.

Opérations sur les Naturels

Décalages à gauche et à droite

- Si x comporte au plus $N-y$ chiffres significatifs⁽¹⁾, alors le décalage à gauche est équivalent à la multiplication par 2^y .

$$x \ll y = x \cdot 2^y$$

- Le décalage à droite est équivalent à la division entière par 2^y .

$$x \gg y = x / 2^y$$

(1) Ce qui est équivalent à $x < 2^{N-y}$. Par conséquent $x \cdot 2^y < 2^{N-y+y} = 2^N$ ce qui implique que $x \cdot 2^y$ est représentable avec N bits.

Opérations sur les Naturels

Exemples en langages C et Java

- Des opérateurs permettant d'effectuer des décalages à gauche et à droite sont proposés par de nombreux langages de programmation, tels que Java, C et C++. Ces opérateurs sont notés “<<” pour le décalage à gauche et “>>” pour le décalage à droite.

C

```
#include <stdio.h>
int main() {
    unsigned char x= 20;
    unsigned char y= x << 3;
    printf("%d\n", y);
    return 0;
}
```

Le programme affiche “160”.

Représentation de x en mémoire:

00010100

Représentation de y en mémoire:

10100000

Java

```
public class Test {
    public static void main(String [] args) {
        byte x= 20;
        byte y= x << 3;
        System.out.println(y);
    }
}
```

Le programme affiche “-96”.

Représentation de x en mémoire:

00010100

Représentation de y en mémoire:

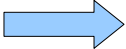
10100000

Table des Matières

Notation des Nombres

- Représentation décimale
- Notation Positionnelle Généralisée
- Changement de Base

Nombres dans un Ordinateur

- Représentation des naturels
 - Addition
 - Soustraction
 - Décalages à gauche et à droite
 -  Multiplication
 - Division
- Représentation des entiers

Opérations sur les Naturels

Multiplication en décimal

- Pour effectuer une multiplication entre deux nombres naturels représentés en décimal, l'algorithme généralement appliqué consiste à effectuer des multiplications intermédiaires.
- Chaque produit intermédiaire est obtenu en multipliant le *multiplicande* (x) par un chiffre du *multiplieur* (y). Le produit final est obtenu en additionnant les résultats des multiplications intermédiaires.

The diagram illustrates the multiplication of 11 (multiplicand x) by 13 (multiplier y). The multiplier 13 is split into its digits 1 and 3. The first intermediate product is 33, calculated as 3×11 . The second intermediate product is 110, calculated as $1 \times 11 \times 10$, where the 11 is circled and a dotted arrow points to a note. The final product 143 is the sum of these two intermediate products.

$$\begin{array}{r} \text{X} \quad \begin{array}{|c|} \hline 11 \\ \hline \end{array} x \\ \quad \begin{array}{|c|} \hline 13 \\ \hline \end{array} y \\ \hline \quad \quad 33 \quad (3 \times 11) \\ + \quad \quad \text{110} \quad (1 \times 11 \times 10) \\ \hline \quad \quad \begin{array}{|c|} \hline 143 \\ \hline \end{array} p \end{array}$$

produits intermédiaires

Note : 11 décalé d'un chiffre vers la gauche

Opérations sur les Naturels

Multiplication en décimal vs en binaire

- On peut écrire plus formellement la multiplication en décimal effectuée par l'algorithme du slide précédent.
- On se rappelle que le multiplicateur y est représenté avec la notation positionnelle.

$$y = \sum_{i=0}^{M-1} y_i \cdot 10^i$$

$$y = \sum_{i=0}^{M-1} y_i \cdot 2^i$$

- Le produit est calculé comme suit

$$p = x \cdot y = \sum_{i=0}^{M-1} \underbrace{y_i \cdot x}_{\text{produits intermédiaires}} \cdot 10^i$$

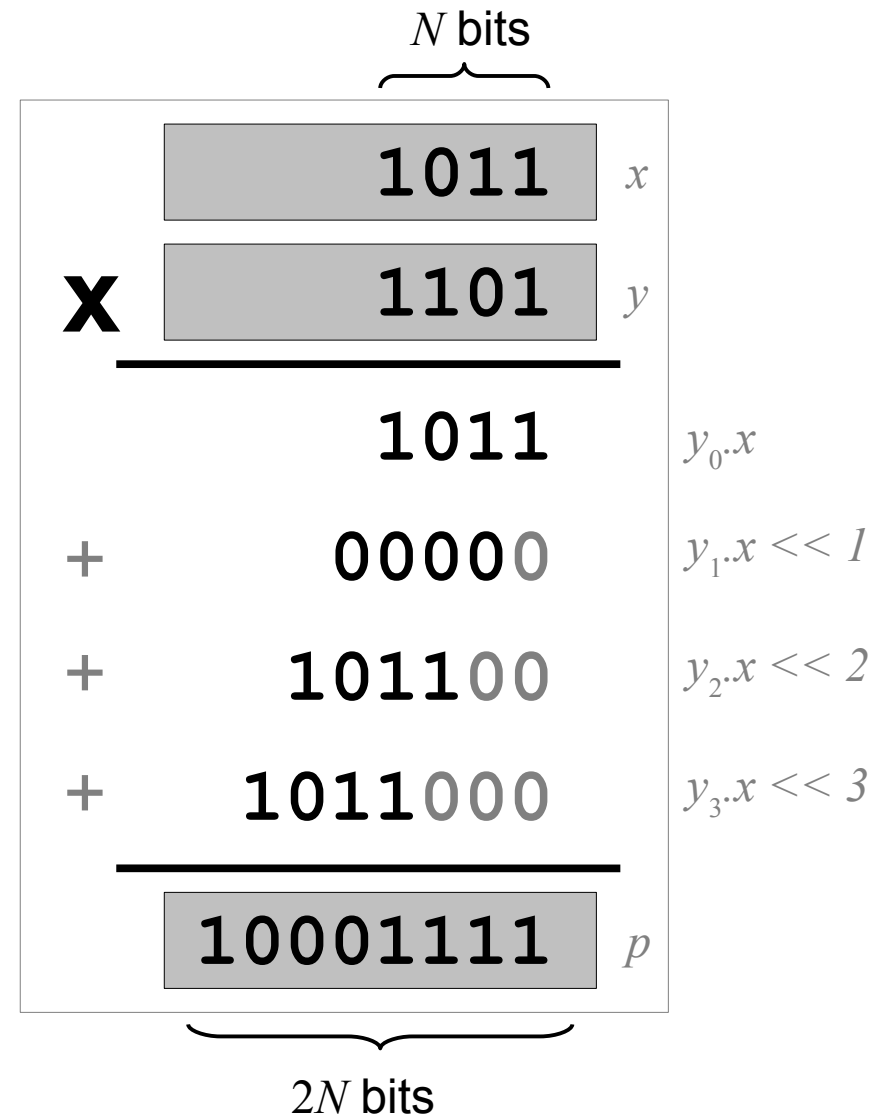
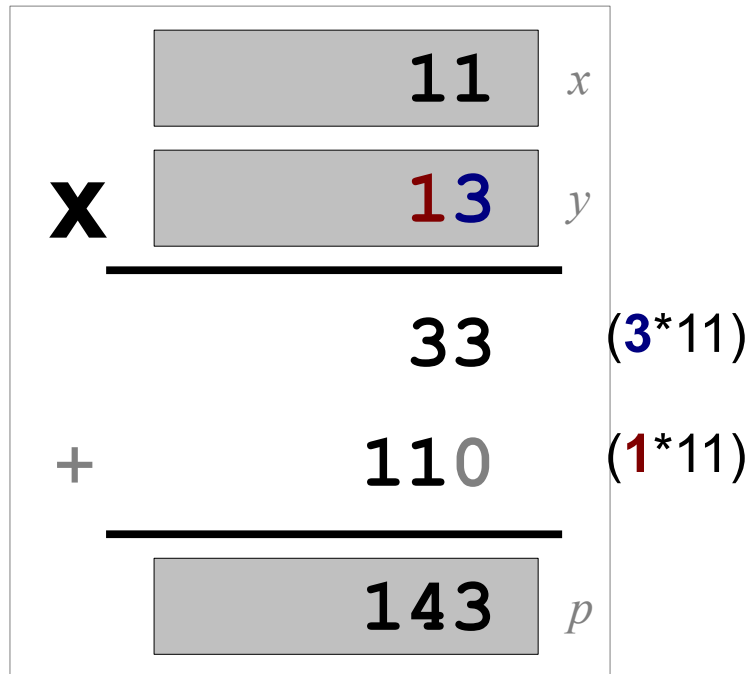
$$p = x \cdot y = \sum_{i=0}^{M-1} y_i \cdot x \cdot 2^i = \sum_{i=0}^{M-1} (y_i \cdot x) \ll i$$

y_i vaut 0 ou 1,
donc $y_i x$ vaut 0 ou x

La multiplication par 10^i ou 2^i correspond à un décalage de i chiffres vers la gauche.

Opérations sur les Naturels

Exemple



dans le pire des cas, le produit sera $(2^N - 1)^2 = 2^{2N} - 2^{N+1} + 1$
ce qui nécessite $2N$ bits pour être représenté.

Opérations sur les Naturels

Accumulation des produits partiels

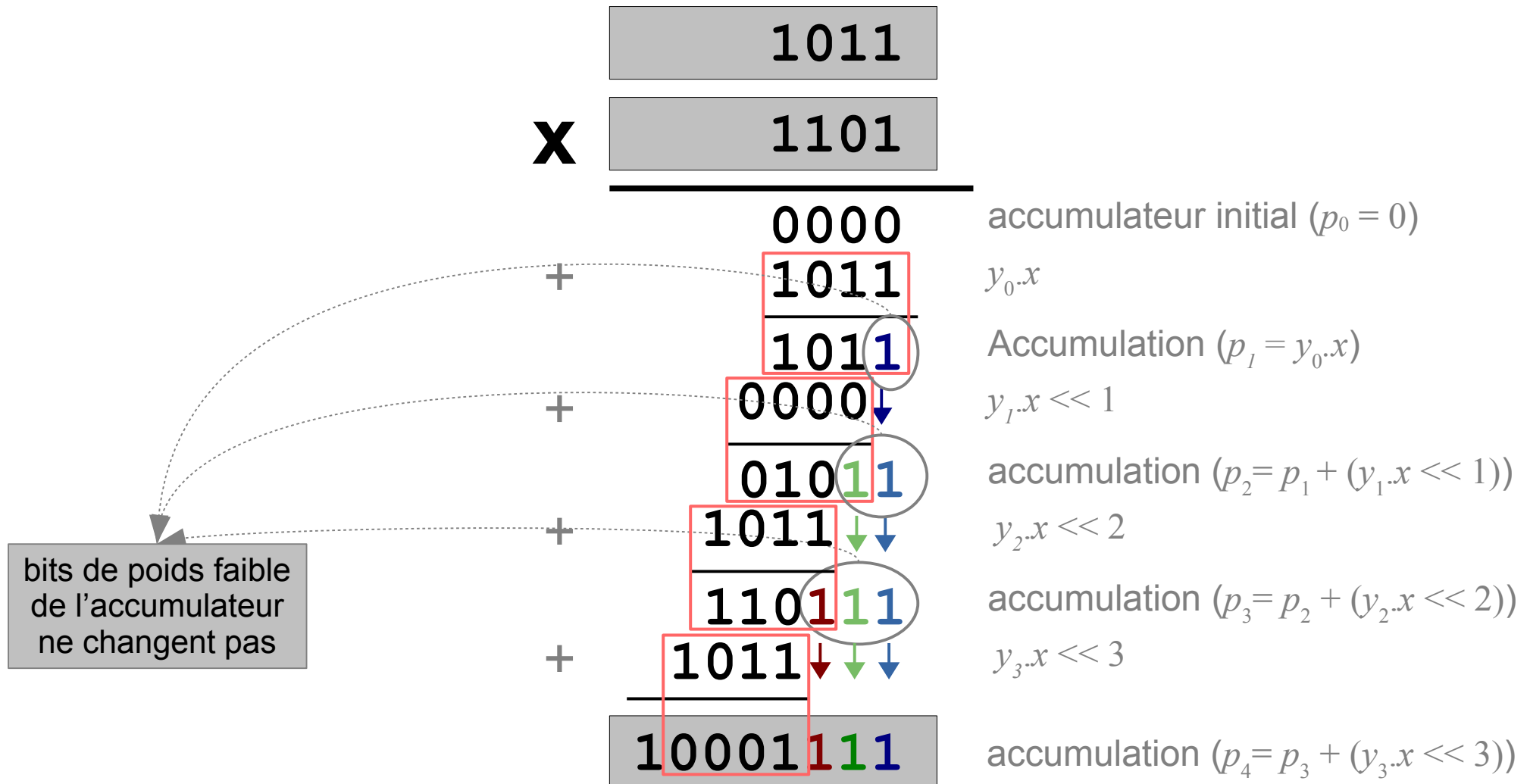
- Il reste un problème avec cet algorithme qui le rend difficile à implémenter sous forme matérielle (électronique). Il faut en effet calculer les produits partiels, les conserver et les additionner ensuite.
- On propose d'additionner les produits partiels au fur et à mesure de leur production dans un accumulateur p . Au début p vaut 0 et en fin d'algorithme il vaudra le produit de x et y .
- A chaque itération i ($0 \leq i < N$), p est mis à jour comme suit

$$p_{i+1} = p_i + ((y_i \cdot x) \ll i)$$

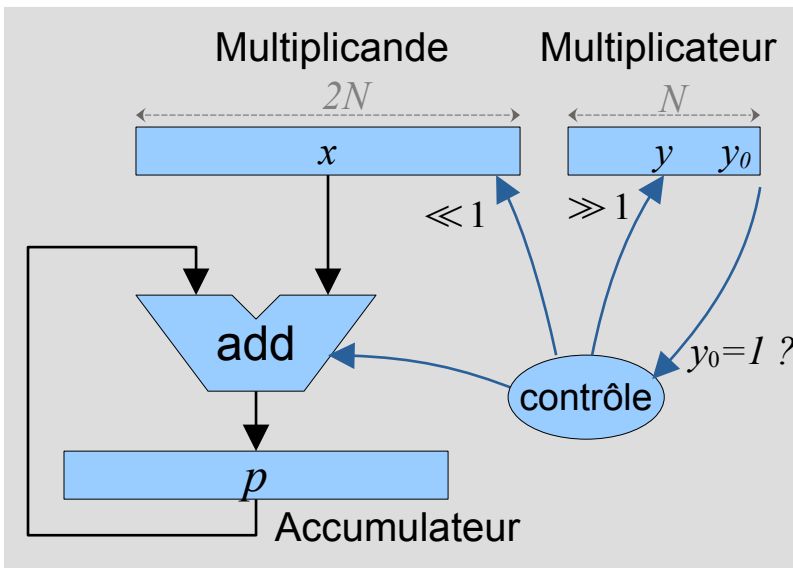
- En raison du décalage par i du produit partiel à l'étape i , les i bits de poids faible restent inchangés lors de l'addition à l'accumulateur. Par conséquent, cette addition peut être réalisée sur seulement N bits.

Opérations sur les Naturels

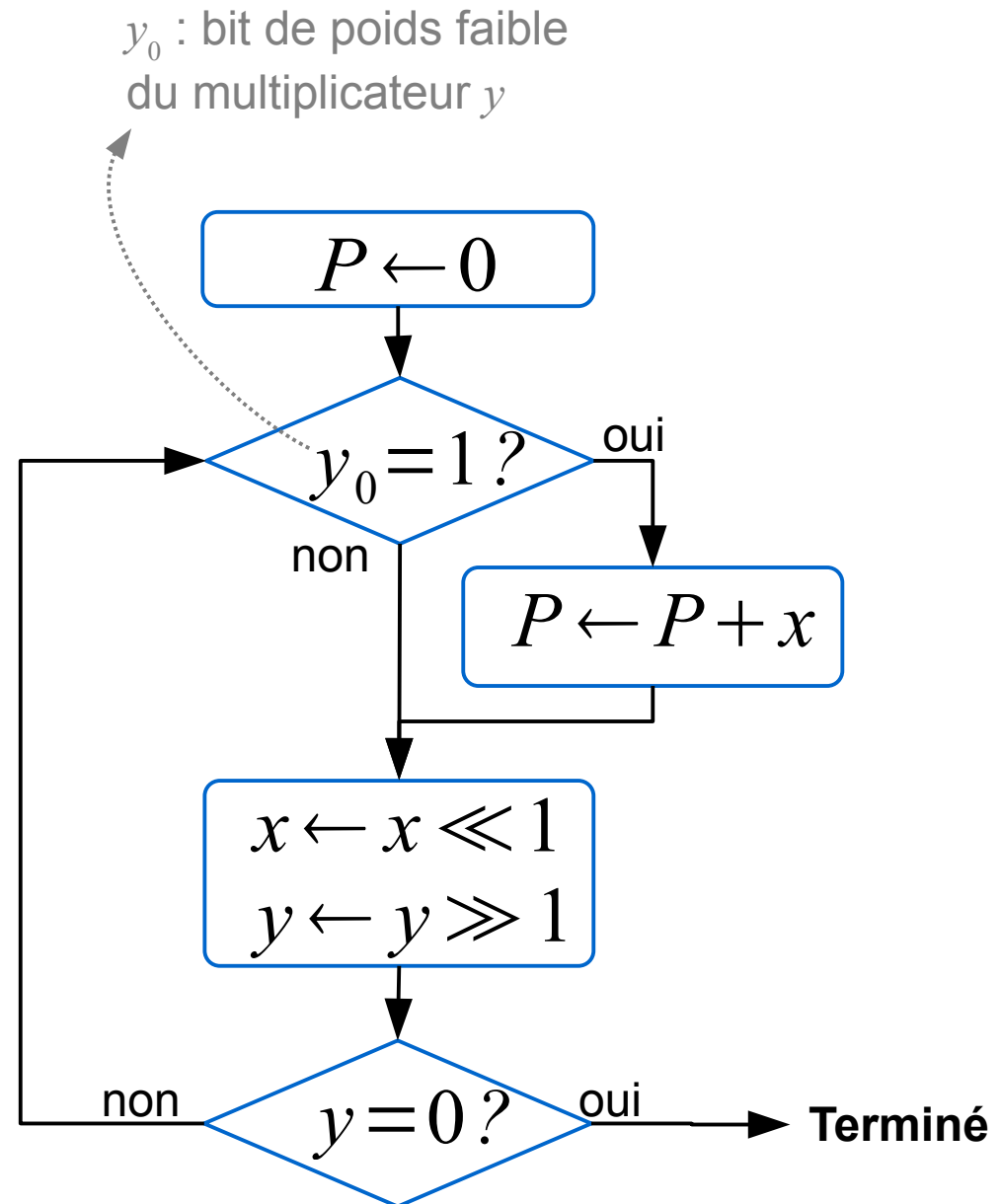
Exemple – accumulation des produits partiels



Opérations sur les Naturels



Explication : x , y et p sont stockés dans des registres. Le registre x peut être décalé vers la gauche ($\ll 1$), y vers la droite ($\gg 1$). Le contrôle de l'algorithme teste si $y_0=1$, effectue l'addition de x et p , décale les registres et répète ces opérations.



Opérations sur les Naturels

Multiplication en binaire

- Exemple : multiplication de $x=7$ (0111) par $y=5$ (0101), $N=4$

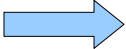
Itération	Multiplicande x	Multiplicateur y	Produit P
Init.	00000111	0101	00000000
1	00000111	010 <u>1</u>	00000000 +00000111
2	00001110	001 <u>0</u>	00000111
3	00011100	000 <u>1</u>	00000111 +00011100
Fin		<u>0000</u>	<u>00100011</u>

Table des Matières

Notation des Nombres

- Représentation décimale
- Notation Positionnelle Généralisée
- Changement de Base

Nombres dans un Ordinateur

- Représentation des naturels
 - Addition
 - Soustraction
 - Décalages à gauche et à droite
 - Multiplication
 -  Division
- Représentation des entiers

Opérations sur les Naturels

Division en décimal

- La division entière en décimal (dite « division longue ») fonctionne suivant un algorithme avec décalage et soustraction successifs.
- Exemple : division de 217 par 11.

$$\begin{array}{r} \boxed{217} \quad / 11 \\ \underline{110} \quad (1 * 11 * 10^1) \\ 107 \\ \underline{99} \quad (9 * 11 * 10^0) \\ \boxed{8} \quad \boxed{19} \\ \text{(reste)} \quad \text{(quotient)} \end{array}$$

Dividende= 217

Divisible par un multiple entier de $11 * 10^1$?

Oui : quotient= **1**, reste= 107.

Nouveau dividende= 107

Divisible par un multiple entier de $11 * 10^0$?

Oui : quotient= **9**, reste= 8.

Reste= 8. Fini.

Opérations sur les Naturels

Division en décimal vs en binaire

- Par définition, la division entière d'un dividende (D) par un diviseur (d) donne un quotient (q) et un reste (r). Elle est exprimée comme :

$$D = d \cdot q + r$$

- Si le quotient est exprimé en représentation positionnelle, on obtient

$$\begin{aligned} D &= d \cdot q + r \\ &= d \cdot \left(\sum_{i=0}^{N-1} q_i \cdot 10^i \right) + r \\ &= \underbrace{q_{N-1} \cdot d \cdot 10^{N-1} + q_{N-2} \cdot d \cdot 10^{N-2} + \dots + q_0 \cdot d \cdot 10^0 + r}_{< d \cdot 10^{N-1}} \end{aligned}$$

- On constate que diviser par $d \cdot 10^{N-1}$ fournit q_{N-1} comme quotient et le **reste de la somme** comme reste.
- En répétant cette division par $d \cdot 10^{N-2}$, $d \cdot 10^{N-3}$, ..., $d \cdot 10^0$, sur le reste obtenu à l'étape précédente, toutes les autres composantes q_{N-2} , ..., q_0 du quotient sont obtenues.

Opérations sur les Naturels

Division en décimal vs en binaire

- Le principe de la division en binaire est identique, à l'exception qu'à chaque étape, la composante q_i du quotient obtenue vaut soit 1 soit 0.

$$\begin{aligned} D &= d \cdot q + r \\ &= d \cdot \left(\sum_{i=0}^{N-1} q_i \cdot 2^i \right) + r \\ &= \underbrace{q_{N-1}}_{0 \text{ ou } 1} \cdot d \cdot 2^{N-1} + \underbrace{q_{N-2} \cdot d \cdot 2^{N-2} + \dots + q_0 \cdot d \cdot 2^0}_{< d \cdot 2^{N-1}} + r \end{aligned}$$

- Conséquences :
 - il ne faut donc pas effectuer de division entière à chaque étape mais simplement **tester si le dividende courant est supérieur à $d \cdot 2^i$** .
 - de plus, $d \cdot 2^i$ est égal à $d \ll i$, ce qui peut être facilement réalisé par une implémentation matérielle.

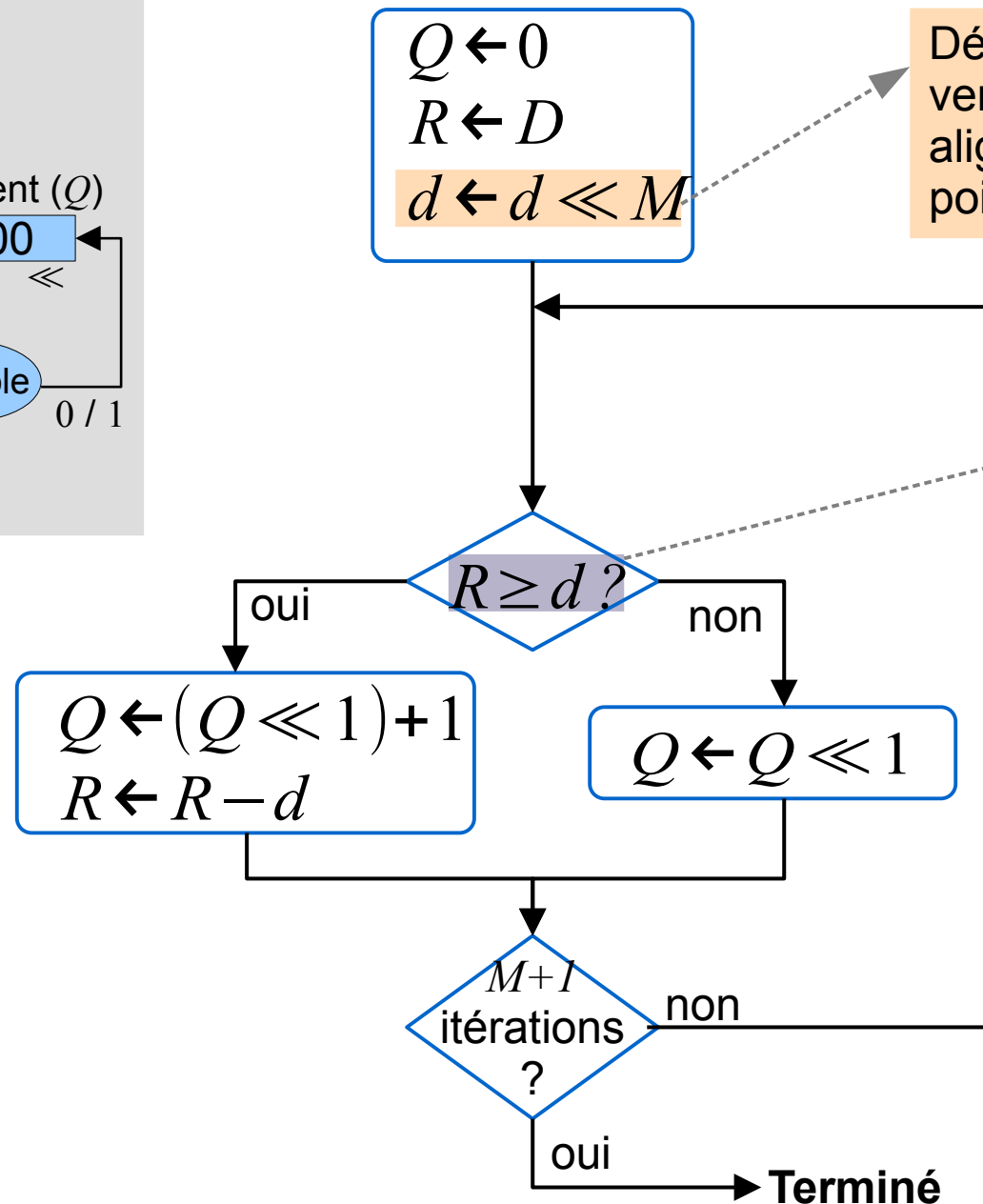
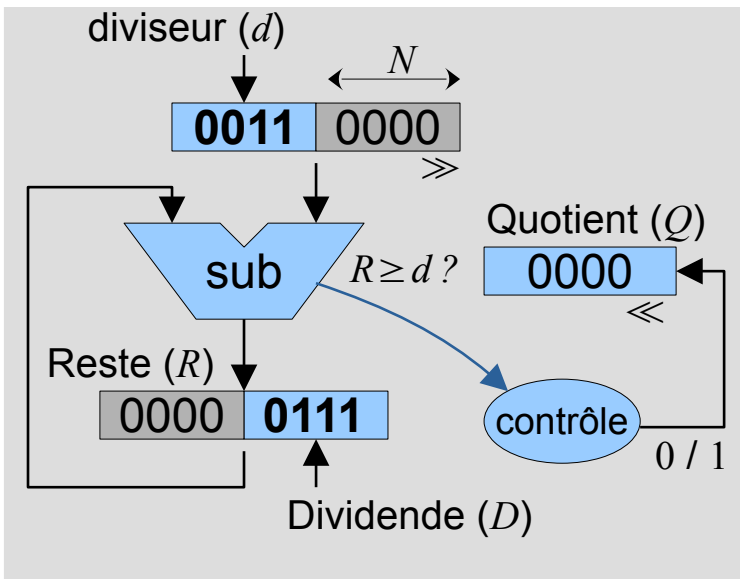
Opérations sur les Naturels

Exemple

- Division de 217 par 11 en binaire.

11011001	/	1011	
1011			(1*1011) << 4
0101			
0000			(0*1011) << 3
1010			
0000			(0*1011) << 2
10100			
1011			(1*1011) << 1
10011			
1011			(1*1011)
1000			
(reste)		10011	(quotient)

Opérations sur les Naturels



Décaler de M positions vers la gauche jusqu'à alignement des bits de poids fort de d et R

Tester $R \geq d$ est typiquement effectué en testant $R - d \geq 0$

Q : Quotient
 R : reste
 D : Dividende
 d : diviseur
 N : largeur (bits)

Opérations sur les Naturels

Division en binaire

- Exemple : division de $D=19$ (010011) par $d=3$ (000011), $N=6$

$M+1$ itérations	Itération	Reste R	diviseur d	$R \geq d$	Quotient q
	Init.	010011	011000 (décalage de $M=3$)		0
	1	010011	011000	non	0
	2	010011 -001100	001100	oui	1
	3	000111 -000110	000110	oui	11
	4	000001	000011	non	110
	Fin	000001			110

Opérations sur les Naturels

Division en binaire

- Exemple : division de $D=51$ (110011) par $d=9$ (001001), $N=6$

Itération	Reste R	diviseur d	$R \geq d$	Quotient q
Init.	110011	100100 (décalage de $M=2$)		0
1	110011 -100100	100100	oui	1
2	001111	010010	non	10
3	001111 -001001	001001	oui	101
Fin	000110			<u>101</u>

Table des Matières

Notation des Nombres

- Notation Décimale Positionnelle
- Notation Positionnelle Généralisée
- Changement de Base

Nombres dans un Ordinateur

- Représentation des naturels

→ Représentation des entiers

Représentation des Entiers

Représentation des entiers

- Le principe de représentation des *entiers* est très similaire à celui de la représentation des *naturels*.
- La différence est qu'il faut pouvoir distinguer les nombres positifs des nombres négatifs. Il faut également adapter les opérations arithmétiques.

Nombres signés

- Pour représenter un nombre négatif en décimal, nous le préfixons avec un symbole particulier, le tiret (-).
- En informatique, d'autres moyens sont utilisés. Nous allons en considérer deux
 - l'utilisation d'un **bit de signe**
 - la représentation en **complément à deux**

Représentation des Entiers

Première tentative – utilisation d'un bit de signe

- Le principe consiste à réserver un bit dans la représentation pour indiquer s'il s'agit d'un nombre positif ou d'un nombre négatif. Par exemple : 0 positif / 1 négatif
 - Si $w_{N-1} = 0$, alors $x \geq 0$. Sinon, $x \leq 0$

$$w = w_{N-1} w_{N-2} \dots w_1 w_0$$

- La valeur du nombre x est alors donnée par l'équation suivante.

$$x = (-1)^{w_{N-1}} \cdot \sum_{i=0}^{N-2} w_i \cdot 2^i$$

- Exemple ($N = 12$) :

$$w = 1\ 10011000011$$

$$(-1)^1 \cdot (1024 + 128 + 64 + 2 + 1) = -1219$$

$$w = 0\ 10011000011$$

$$(-1)^0 \cdot (1024 + 128 + 64 + 2 + 1) = 1219$$

Représentation des Entiers

Première tentative – utilisation d'un bit de signe

- **Problème (1)** : l'utilisation d'un bit de signe donne lieu à l'existence de 2 représentations différentes du nombre zéro.

$w = 0\ 0 \dots 00$ représente $x = +0$

$w = 1\ 0 \dots 00$ représente $x = -0$

- **Problème (2)** : l'addition binaire ne fonctionne pas toujours
 - Exemple ($N=4$)
 - $x = 2$ $\rightarrow$ représenté par 0010
 - $-x = -2$ $\rightarrow$ représenté par 1010
 - addition directe des représentations donne $\begin{array}{r} 0010 \\ 1010 \\ \hline 1100 \end{array} (-4)$
 - Il serait possible, en fonction des signes des nombres à additionner, de travailler avec les valeurs absolues des nombres et de faire selon les cas des additions ou soustractions. Cependant, cela est moins facile à implémenter sous forme matérielle.

Représentation des Entiers

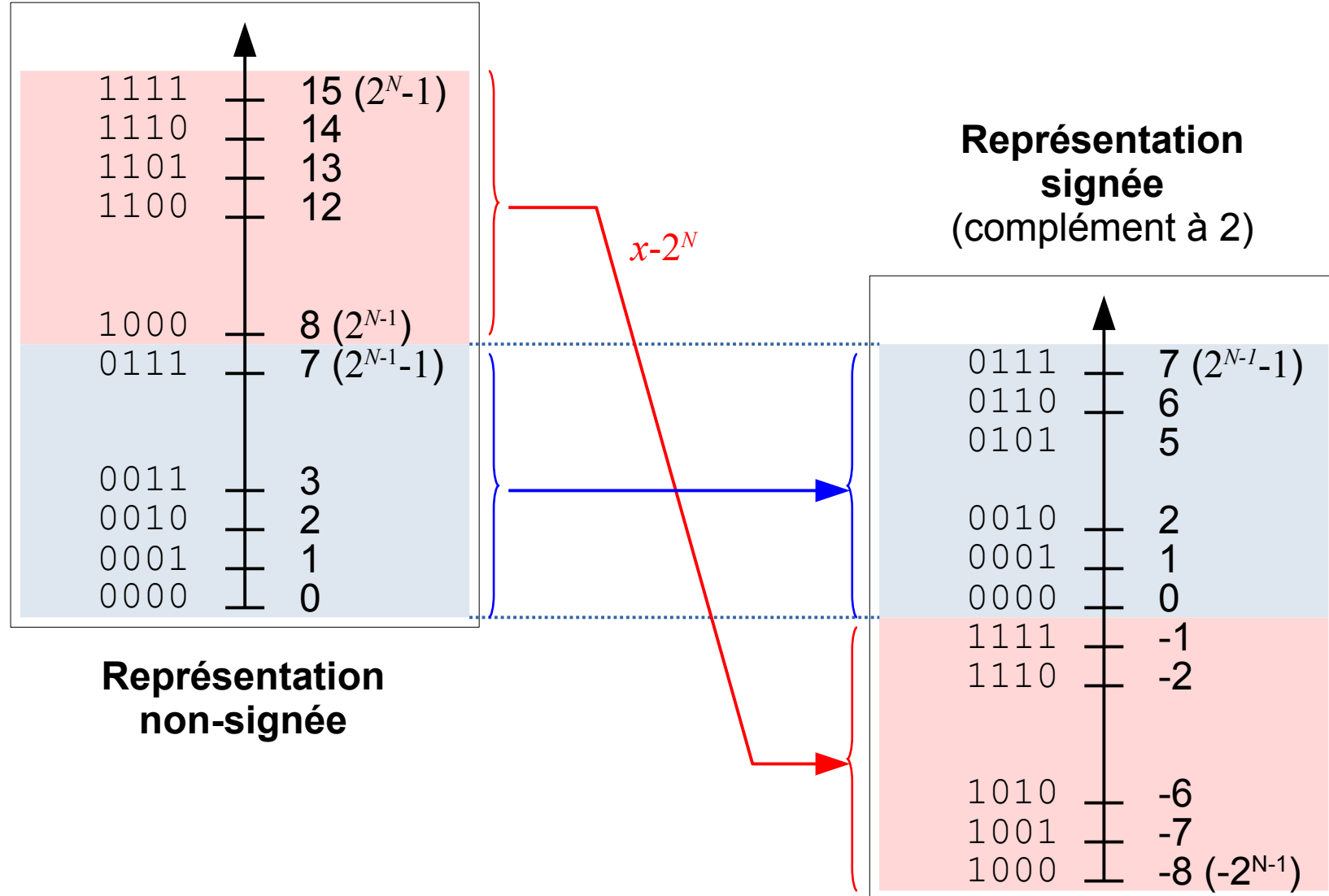
Seconde tentative – complément à 2

- Objectif : trouver une représentation dans laquelle l'addition binaire directe des représentations d'un nombre et de son opposé (son complément) donne 0.
- Il existe différents systèmes possédant cette propriété. Le plus répandu est la **représentation en complément à 2** (*two's complement*). Elle est utilisée dans la plupart des processeurs aujourd'hui.
- Il existe d'autres représentations possédant cette propriété, comme le “complément à 1” (*ones' complement*) que nous n'aborderons pas ici ¹.

(1) Note : nous aurons la chance (!) d'en parler lors du cours de Réseaux I.

Représentation des Entiers

Seconde tentative – complément à 2



Représentation des Entiers

Seconde tentative – complément à 2

- Définition : sur N bits, la représentation de l'opposé (le complément) d'un nombre x ($0 < x < 2^{N-1}$) est donnée par la représentation non signée de $2^N - x$.
- Exemple ($N=4$) :
 - $x = 2 \rightarrow$ représenté par $w = 0010$
 - $-x = -2 \rightarrow$ représentation de $2^N - x = 14$ par $w = 1110$
- Propriétés :
 - On peut vérifier que $(x + 2^N - x) \bmod 2^N = 0$
 - Le complément du complément de x est $2^N - (2^N - x) = x$
 - Le complément de 0 est $(2^N - 0) \bmod 2^N = 0$
- Exemple :
 - $2 + (-2)$ peut être calculé en additionnant les représentations

$$\begin{array}{r} 0010 \\ + 1110 \\ \hline 0000 \end{array}$$

Représentation des Entiers

Représentation en complément à 2

- Dans la représentation en **complément à 2** (*two's complement*), un entier x est représenté par un mot w de N bits et le bit de poids fort a un poids négatif

$$w = w_{N-1} w_{N-2} \dots w_1 w_0$$

- Le nombre x est obtenu par

$$x = w_{N-1} \cdot (-2^{N-1}) + \sum_{i=0}^{N-2} w_i \cdot 2^i$$

↓
poids négatif

- Exemple ($N=12$) :

– L'entier -1219 est représenté par $w = 1\,01100111101$

$$\begin{aligned} x = -1219 &= -2048 + 512 + 256 + 32 + 16 + 8 + 4 + 1 \\ &= -2^{11} + 2^9 + 2^8 + 2^5 + 2^4 + 2^3 + 2^2 + 2^0 \end{aligned}$$

Représentation des Entiers

Représentation en complément à 2

- L'intervalle de nombre représentables avec N bits est

$$\left[-2^{N-1}, 2^{N-1} - 1 \right]$$

- Exemples:
 - Exemples en langage C et Java

N	Type		Intervalle de nombres
	C	Java	
8	char	byte	[-128, 127]
16	short	short	[-32768, 32767]
32	int	int	[-2147483648, 2147483647]
64	long int	long	[-9223372036854775808, 9223372036854775807]

Note : La borne inférieure est obtenue avec le bit de poids fort à 1 et les autres à 0. La borne supérieure est obtenue comme le nombre maximum représentable en non-signé avec $N-1$ bits.

$$\sum_{i=0}^{N-2} 2^i = 2^{N-1} - 1$$

Représentation des Entiers

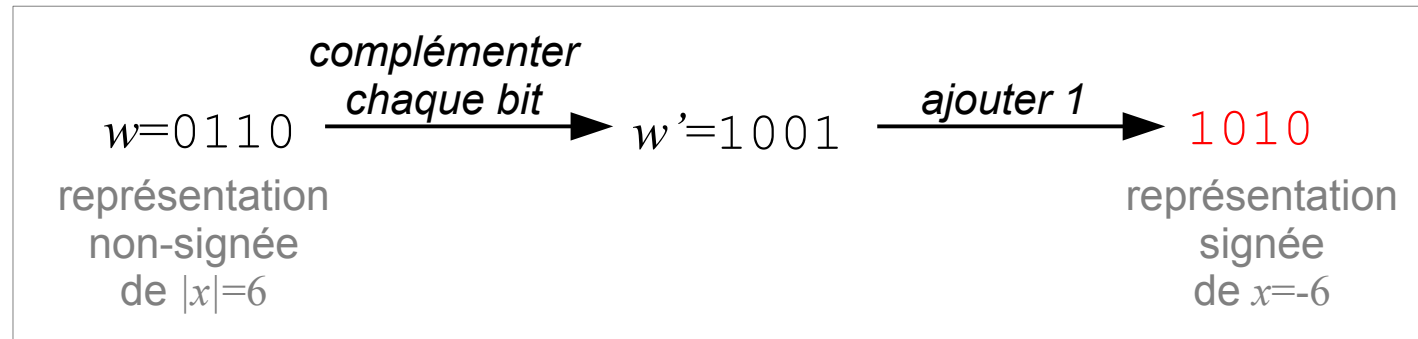
Représentation en complément à 2

- Voici une astuce pour obtenir facilement la représentation en complément à 2 sur N bits d'un nombre négatif x , avec $-2^{N-1} < x < 0$

- 1) Obtenir la représentation non-signée, w , de $|x|$
- 2) Déterminer w' en complémentant chaque bit de w
- 3) Ajouter 1 à w'

- Exemple ($N = 4$) :

- comment représenter $x = -6$?



Note : pourquoi cela fonctionne-t-il ? $2^N - |x| = ((2^N - 1) - |x|) + 1$

$$(2^N - 1) - |x| = \left(\sum_{i=0}^{N-1} 2^i \right) - x = \sum_{i=0}^{N-1} (1 - w_i) \cdot 2^i \quad 1 - w_i = w'_i, \forall 0 \leq i < N$$

Représentation des Entiers

Taille de mot

- Quelle taille de mot minimum, N , faut-il utiliser pour représenter un entier x donné ?

- Pour $x \geq 0$, il faut trouver N (naturel) tel que

$$2^{N-2}-1 < x \leq 2^{N-1}-1$$

$$N = \lceil \log_2(x+1) + 1 \rceil$$

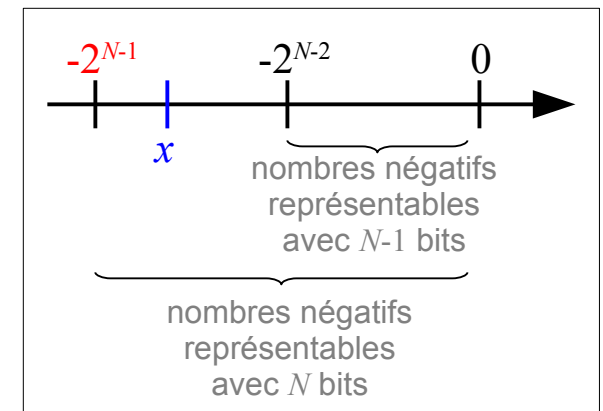
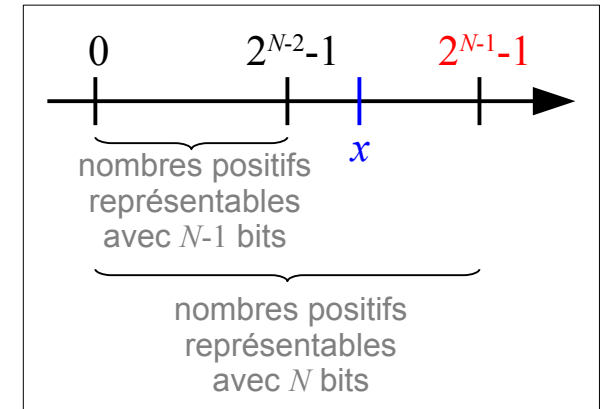
- Pour $x < 0$, il faut trouver N (naturel) tel que

$$-2^{N-1} \leq x < -2^{N-2}$$

$$N = \lceil \log_2(-x) + 1 \rceil$$

- Exemples:

- $x = 234 \rightarrow N = \lceil \log_2(235) + 1 \rceil = 9$
- $x = -39 \rightarrow N = \lceil \log_2(39) + 1 \rceil = 7$



Représentation des Entiers

Types des variables et transtypage

- Dans un langage de programmation tel que Java, C ou C++, le type d'une variable est intimement lié à la représentation du contenu de la variable en mémoire.
- Par exemple
 - en Java, type `int` → 32 bits, complément à 2
type `byte` → 8 bits, complément à 2
 - en C et en C++, type `char` → 8 bits, complément à 2
type `unsigned char` → 8 bits, non signé
- Il est important de savoir à quoi correspond un type de variable, par exemple lorsque l'on utilise des opérations comme le **transtypage**, i.e. lorsque l'on interprète le contenu d'une variable avec un type différent.

Représentation des Entiers

Transtypage en Java et en C

```
public class Test {  
    public static void main(String [] args) {  
        int x1= 243;  
        byte x2= (byte) x1;  
        System.out.println(x2);  
    }  
}
```

$$x_1 = -w_{31} \cdot 2^{31} + \sum_{i=0}^{30} w_i \cdot 2^i = 243$$

Le programme affiche "-13".

x1 en mémoire: 0000000000000000000000000000000011110011

x2 en mémoire: 11110011

```
#include <stdio.h>  
int main() {  
    char x_c2= -13;  
    unsigned char x_ns= (unsigned char) x_c2;  
    printf("%d\n", x_ns);  
    return 0;  
}
```

$$x_2 = -w_7 \cdot 2^7 + \sum_{i=0}^6 w_i \cdot 2^i = -13$$

Le programme affiche "243".

x_c2 en mémoire: 11110011

x_ns en mémoire: 11110011

$$x_{c2} = -w_7 \cdot 2^7 + \sum_{i=0}^6 w_i \cdot 2^i = -13$$

$$x_{ns} = \sum_{i=0}^7 w_i \cdot 2^i = 243$$

Table des Matières

Notation des Nombres

- Notation Décimale Positionnelle
- Notation Positionnelle Généralisée
- Changement de Base

Nombres dans un Ordinateur

- Représentation des naturels
- Représentation des entiers



Addition

- Opposé, soustraction
- Décalages
- Multiplication
- Extension signée

Opérations sur les Entiers

Addition en complément à 2

- L'addition de nombres en complément à deux est effectuée de la même façon que l'addition de nombres naturels (non-signés).
- Cela fonctionne car l'addition est effectuée modulo 2^N .
- Exemples ($N=4$)

$$\begin{array}{r} 111 \\ 0111 \text{ (7)} \\ + 1010 \text{ (-6)} \\ \hline 10001 \text{ (1)} \end{array}$$

$$\begin{array}{r} 0100 \text{ (4)} \\ + 1001 \text{ (-7)} \\ \hline 1101 \text{ (-3)} \end{array}$$

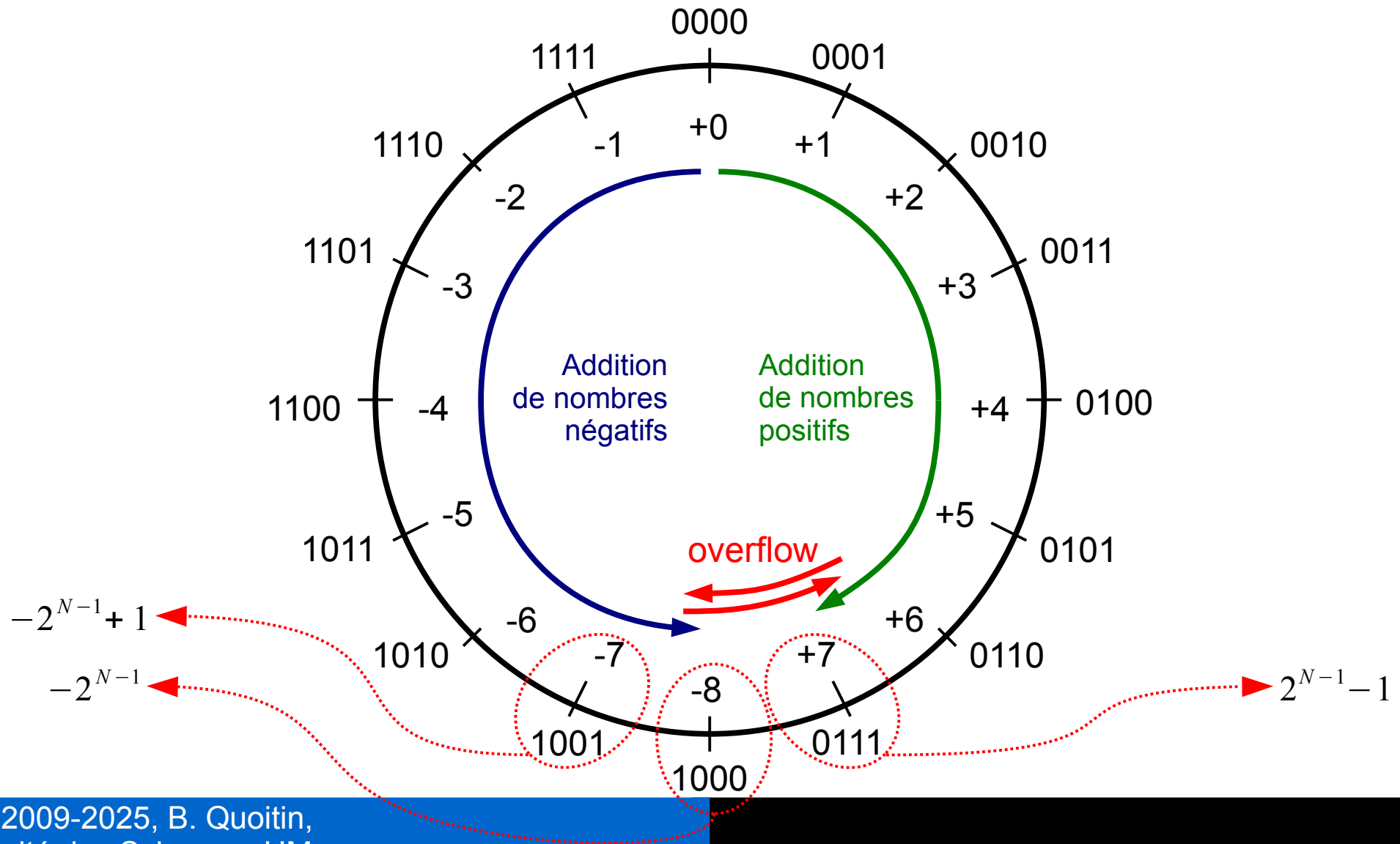
$$\begin{array}{r} 111 \\ 1110 \text{ (-2)} \\ + 1010 \text{ (-6)} \\ \hline 11000 \text{ (-8)} \end{array}$$

$$\begin{array}{r} 11 \\ 0110 \text{ (6)} \\ + 1101 \text{ (-3)} \\ \hline 10011 \text{ (3)} \end{array}$$

le bit supplémentaire
disparaît avec le modulo 2^N

Opérations sur les Entiers

Dépassements – cas $N=4$



Opérations sur les Entiers

Dépassement en complément à 2

- Il y a dépassement lors de l'addition en complément à 2 si et seulement si les deux conditions suivantes sont vraies
 - x et y sont de même signe
 - la somme est de signe différent

> 0	< 0		pas de dépassement
< 0	> 0		pas de dépassement
> 0	> 0	< 0	dépassement
> 0	> 0	> 0	pas de dépassement
< 0	< 0	> 0	dépassement
< 0	< 0	< 0	pas de dépassement

- Exemples ($N=4$)

$$\begin{array}{r} 6 \\ + -3 \\ \hline 3 \end{array} \quad \begin{array}{r} 0110 \\ + 1101 \\ \hline 10011 \end{array}$$

Pas d'overflow :-)

$$\begin{array}{r} 5 \\ + 6 \\ \hline 11 \end{array} \quad \begin{array}{r} 0101 \\ + 0110 \\ \hline 1011 \end{array} \quad (-5)$$

Overflow :-)

Opérations sur les Entiers

Dépassement en complément à 2

- Explication : un dépassement n'est possible que si les 2 nombres sont de même signe (**vérifiez !**). Ensuite, on peut traiter séparément les cas où x et y sont tous les deux positifs ou tous les deux négatifs.
- Cas x et y positifs (cas négatif similaire, **vérifiez!**):

$$0 \leq x, y \leq 2^{N-1} - 1$$

$$0 \leq x+y \leq 2 \cdot (2^{N-1} - 1) = 2^N - 2$$

- Il y a dépassement lorsque $x+y > 2^{N-1} - 1$, donc l'intervalle des résultats qui correspondent à un dépassement est

$$2^{N-1} \leq x+y \leq 2^N - 2$$

- Ces valeurs de la somme $x+y$ ont un bit de poids fort qui vaut 1 alors que les bits de poids fort de x et y valent 0.

Opérations sur les Entiers

Addition en complément à 2

- L'addition en complément à 2, notée $+_{c2}$, peut être résumée comme suit. Sur l'intervalle d'entiers $[-2^{N-1}, 2^{N-1}-1]$, elle forme un groupe commutatif.

$$x +_{c2} y = \begin{cases} x + y - 2^N & \text{si } x + y \geq 2^{N-1} \\ x + y & \text{si } -2^{N-1} \leq x + y < 2^{N-1} \\ x + y + 2^N & \text{si } x + y < -2^{N-1} \end{cases}$$

- Exemples avec $N=4$

- $x = 3$ $y = 4$ $x + y = 7$ $x +_{c2} y = 7$ (cas 2)
- $x = 5$ $y = 6$ $x + y = 11$ $x +_{c2} y = -5$ (cas 1)
- $x = -8$ $y = -8$ $x + y = -16$ $x +_{c2} y = 0$ (cas 3)

Table des Matières

Notation des Nombres

- Notation Décimale Positionnelle
- Notation Positionnelle Généralisée
- Changement de Base

Nombres dans un Ordinateur

- Représentation des naturels
- Représentation des entiers
 - Addition
 - Opposé, soustraction
 - Décalages
 - Multiplication
 - Extension signée

Opérations sur les Entiers

Opposé d'un nombre en complément à 2

- Pour rappel, si un nombre x a la représentation w , alors la représentation de $-x$ est obtenue en complémentant tous les bits de w et en y ajoutant 1.
- Exemple ($N=4$) :
 - $x=-5$ est représenté par $w=1011 \rightarrow 0100$
 $\rightarrow w'=0101$ représente $-x=5$
- Tous les nombres de l'intervalle $[-2^{N-1}, 2^{N-1}-1]$ ont leur opposé dans le même intervalle, **à l'exception de -2^{N-1}** . Il s'agit d'une forme de dépassement de capacité.
- Exemple ($N=4$) :
 - l'opposé de -8 (1000) n'appartient pas à $[-8, 7]$
 - si on applique l'approche ci-dessus, on obtient 0111 en complémentant chaque bit, puis 1000 lorsqu'on ajoute 1.

Opérations sur les Entiers

Soustraction en complément à 2

- La soustraction en complément à 2 se repose sur l'addition en complément à 2. Soustraire y de x revient à additionner x et $-y$.

$$\begin{aligned}x - y &= x + (-y) \\&= x + (\tilde{y} + 1) \\&= x + \tilde{y} + 1\end{aligned}$$

Complémenter
les bits de y

Report initial de 1

- Exemple (N=4) :

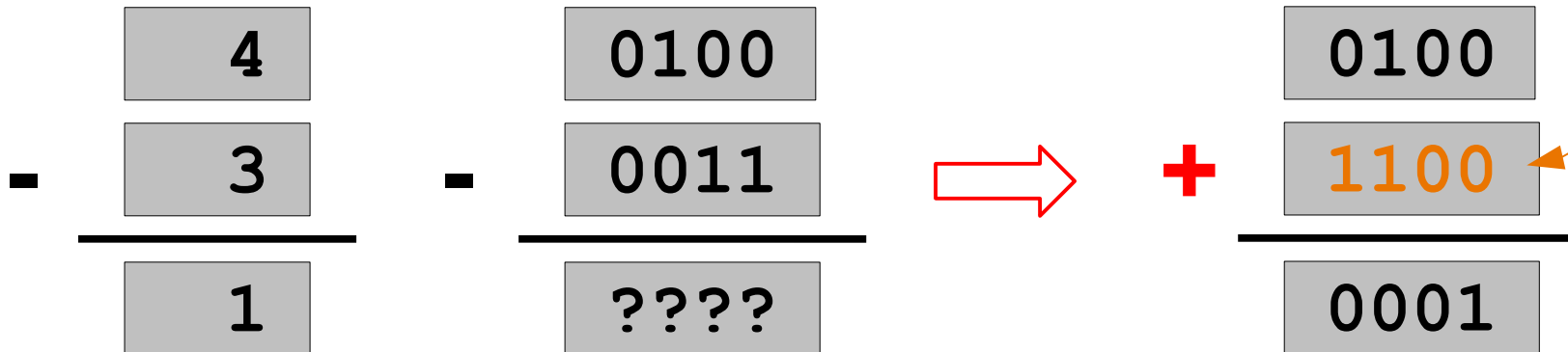


Table des Matières

Notation des Nombres

- Notation Décimale Positionnelle
- Notation Positionnelle Généralisée
- Changement de Base

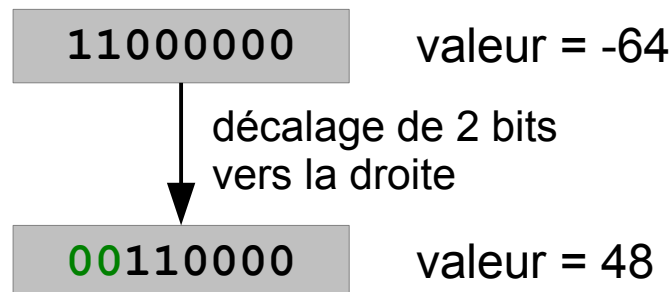
Nombres dans un Ordinateur

- Représentation des naturels
- Représentation des entiers
 - Addition
 - Opposé, soustraction
 - ➡ Décalages
 - Multiplication
 - Extension signée

Opérations sur les Entiers

Décalages

- Avec la représentation en complément à 2, des précautions supplémentaires sont nécessaires lorsque l'on effectue des décalages vers la droite.
- En effet, de nouveaux bits de poids fort qui valent 0 sont ajoutés. Si le nombre décalé est négatif (bit de poids fort à 1), le décalage à droite rend le résultat positif !
- Exemple ($N=8$) :

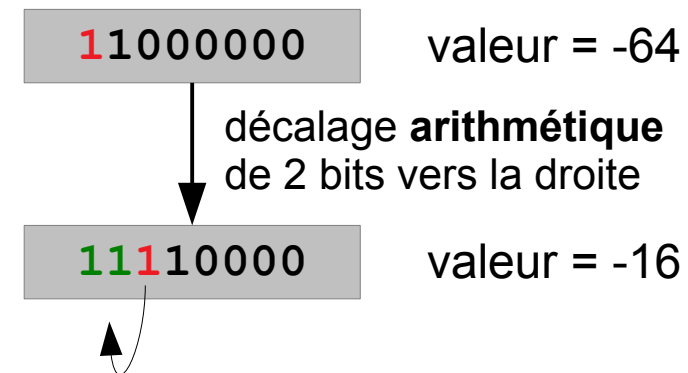
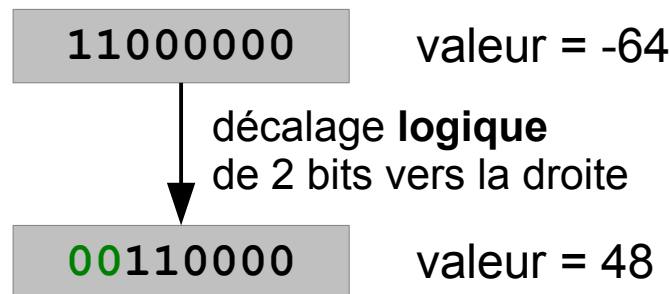


Opérations sur les Entiers

Décalages

- On distingue par conséquent deux types de décalages vers la droite
 - décalage logique (ou décalage non-signé): décale les bits sans se soucier du signe → les nouveaux bits de poids fort valent 0
 - décalage arithmétique: conserve le signe → les nouveaux bits de poids fort sont égaux au "bit de signe" du nombre initial

- Exemples



Opérations sur les Entiers

Décalages en Java

- Le langage Java offre deux opérateurs distincts pour le décalage vers la droite
 - logique = “>>>”
 - arithmétique = “>>”

```
public class Test {  
  
    public static void main(String [] args) {  
        int x= -63;  
        int y= x >> 3;  
        int z= x >>> 3;  
        System.out.println(y);  
        System.out.println(z);  
    }  
}
```

Le programme affiche “-8” et “536870904”

Représentation de x en mémoire:

11111111111111111111111111111000001

Représentation de y en mémoire:

11111111111111111111111111111000

Représentation de z en mémoire:

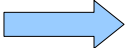
00011111111111111111111111111000

Table des Matières

Notation des Nombres

- Notation Décimale Positionnelle
- Notation Positionnelle Généralisée
- Changement de Base

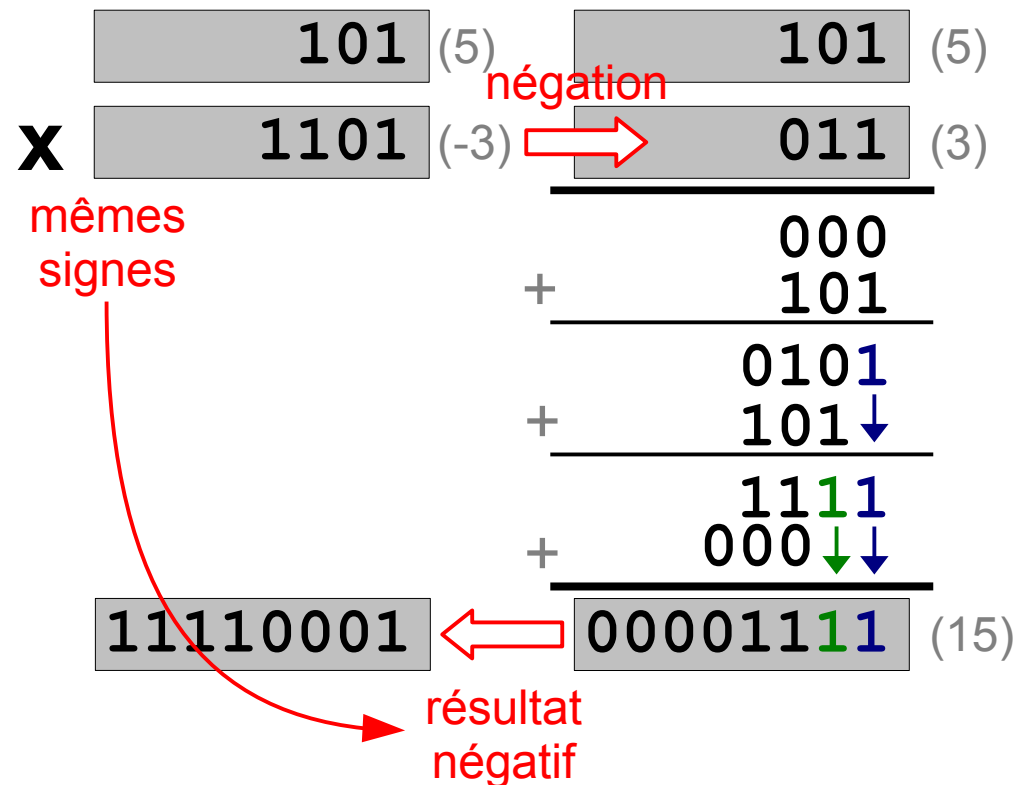
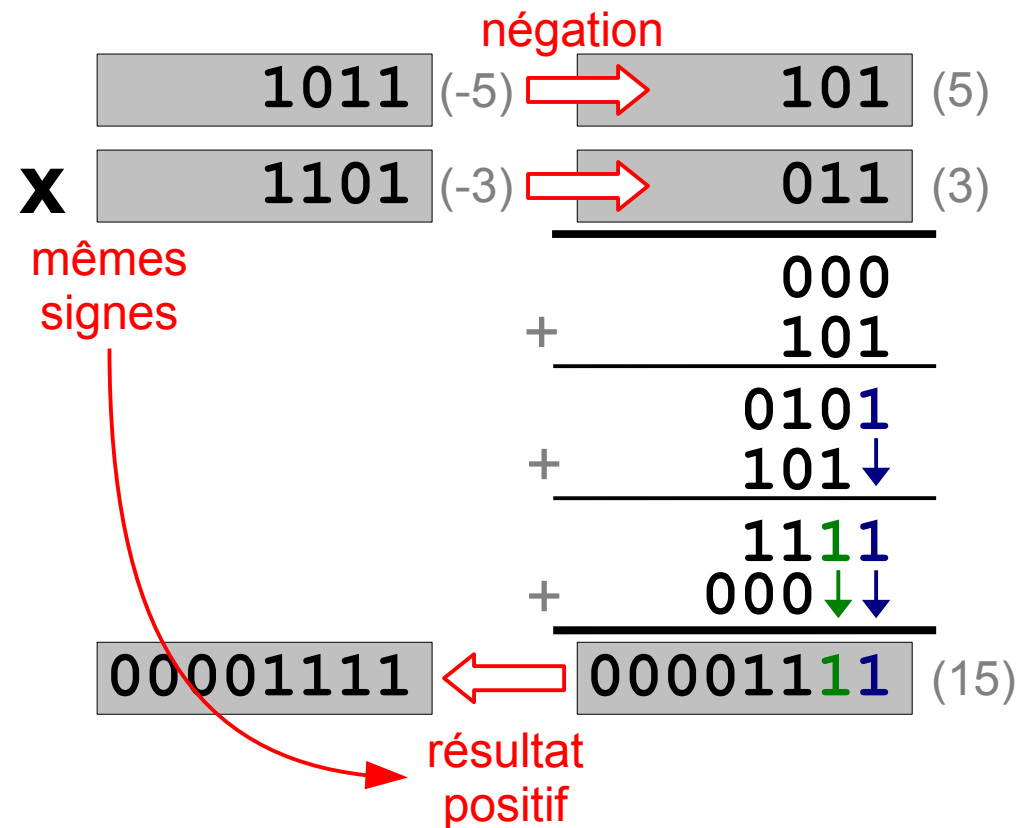
Nombres dans un Ordinateur

- Représentation des naturels
- Représentation des entiers
 - Addition
 - Opposé, soustraction
 - Décalages
 -  Multiplication
 - Extension signée

Opérations sur les Entiers

Multiplication en complément à 2

- Une façon d'effectuer la multiplication signée consiste à procéder **en valeur absolue**, puis de déterminer le signe du résultat sur base des signes des arguments : si $\text{signe}(x) = \text{signe}(y)$ alors résultat positif, sinon négatif



Opérations sur les Entiers

Multiplication en complément à 2

- En fait, il est possible de multiplier directement 2 nombres représentés en complément à deux sur N bits avec la multiplication non-signée si on ne s'intéresse qu'aux N bits de poids faible du résultat (i.e. résultat modulo 2^N).
- Soit x et y deux mots de N bits.
- Soit x_{c2} et y_{c2} , les valeurs de resp. x et y en complément à 2.
- Soit x_{ns} et y_{ns} les valeurs de resp. x et y en non signé.

$$\begin{aligned}(x_{ns} \cdot y_{ns}) \bmod 2^N &= \left\{ (x_{c2} + x_{N-1} \cdot 2^N) \cdot (y_{c2} + y_{N-1} \cdot 2^N) \right\} \bmod 2^N \\ &= \left\{ x_{c2} \cdot y_{c2} + (x_{N-1} \cdot y_{c2} + x_{c2} \cdot y_{N-1}) \cdot 2^N + x_{N-1} \cdot y_{N-1} \cdot 2^N \right\} \bmod 2^N \\ &= (x_{c2} \cdot y_{c2}) \bmod 2^N\end{aligned}$$

Ces termes disparaissent avec le modulo 2^N

Opérations sur les Entiers

Multiplication en complément à 2

- Exemple ($N=5$) :

$$\begin{array}{r} \mathbf{X} \quad \begin{array}{l} \boxed{11011} \quad (-5) \\ \boxed{11101} \quad (-3) \end{array} \\ \hline 11011 \\ 11011 \\ \hline 10000111 \\ 11011 \\ \hline 101011111 \\ 11011 \\ \hline \boxed{11000} \boxed{01111} \quad (15) \end{array}$$

$$\begin{array}{r} \begin{array}{l} \boxed{11101} \quad (-3) \\ \boxed{00101} \quad (5) \end{array} \\ \hline 11101 \\ 11101 \\ \hline \boxed{100} \boxed{10001} \quad (-15) \end{array}$$

disparaissent
avec le modulo 2^5

Table des Matières

Notation des Nombres

- Notation Décimale Positionnelle
- Notation Positionnelle Généralisée
- Changement de Base

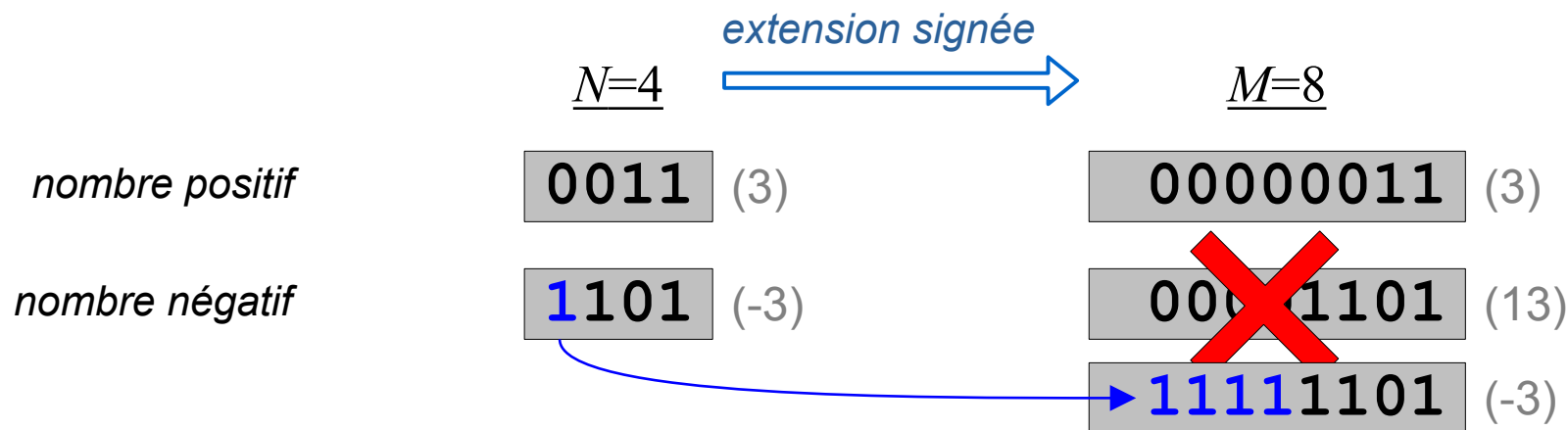
Nombres dans un Ordinateur

- Représentation des naturels
 - Représentation des entiers
 - Addition
 - Opposé, soustraction
 - Décalages
 - Multiplication
- ➡ Extension signée

Opérations sur les Entiers

Extension signée

- Il est souvent nécessaire de copier la représentation d'un nombre vers un mot plus large. Des précautions sont nécessaires lorsque ce nombre est négatif.
- Exemple :
 - copie d'une variable de $N=4$ bits vers une variable de $M=8$ bits.

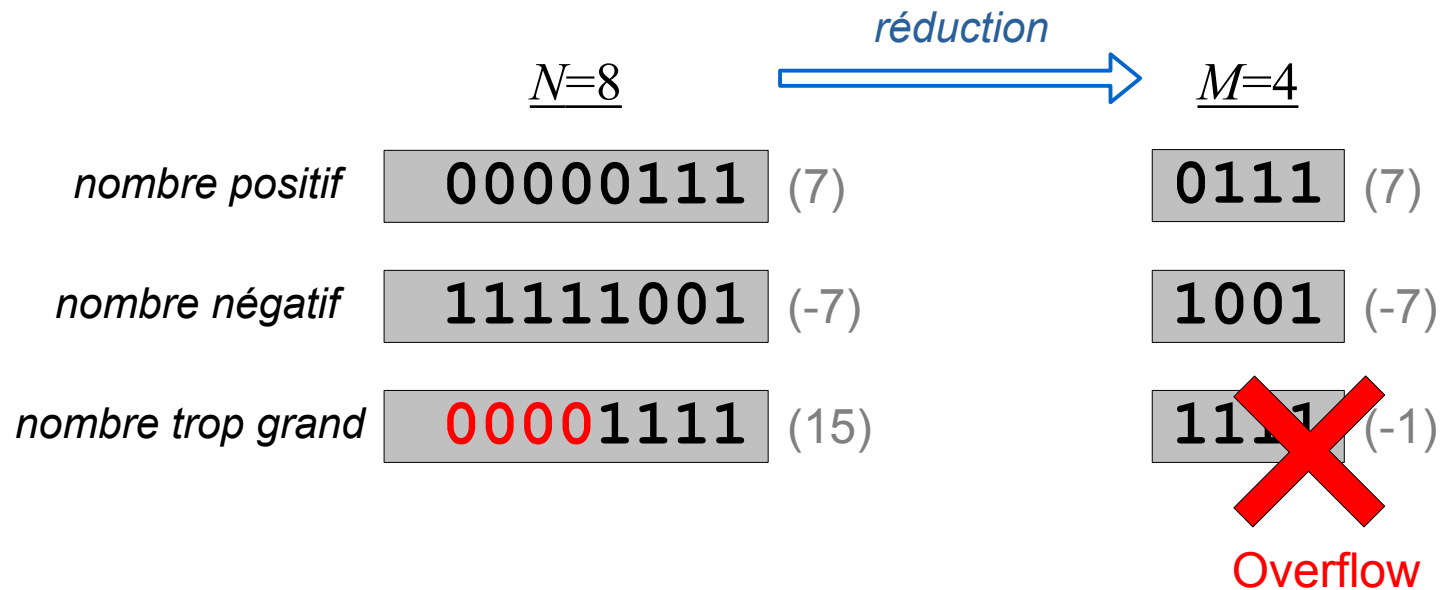


- La règle qui s'applique est la suivante: les bits de poids fort du mot de taille M sont initialisés avec la valeur du bit de poids fort du mot de taille N .

Opérations sur les Entiers

Extension signée

- Pour une copie vers un mot moins large, il suffit de ne garder que les bits de poids faible. Il faut cependant faire attention aux dépassements de capacité !
- Exemple :
 - copie d'une variable de $N=8$ bits vers une variable de $M=4$ bits.



References

- **Computer Organization and Design: The Hardware/Software Interface**, 4th Edition, D. Patterson and J. Hennessy, Morgan-Kaufmann, 2009
- **Digital Design: Principles and Practices** (3rd Edition), J. Wakerly, Prentice Hall, 2001
- **Computer Systems: A Programmer's Perspective** (2nd edition), R. E. Bryant and D. R. O'Hallaron, Addison-Wesley, 2010
- **What Every Computer Scientist Should Know About Floating-Point Arithmetic**, D. Goldberg, ACM Computing Surveys, Vol 23, No 1, March 1991
- **A Practical Introduction to Computer Architecture**, D. Page, Springer, 2009

Remerciements

Merci à toutes les personnes qui ont permis par leurs remarques de corriger et d'améliorer ces notes de cours.